POSTS AND TELECOMMUNICATIONS INSTITUTE OF TECHNOLOGY
Faculty of Information Technology

Intelligent System Development

ASSIGNMENT 07

Memory in Deep Recurrent Networks: a Function-Composition Study of 5-Layer RNN and LSTM in PyTorch and Keras, on Review Sentiment and Stock Volatility

Instructor / Lecturer: Assoc. Prof. Dinh Que Tran, Ph.D.
Email: quetd@ptit.edu.vn, tdque@yahoo.com

Student Name: Bùi Nguyên Hoàng Việt
Student ID: B23DCDT285
Semester: I.2025 – 2026
Submission: Individual Project
Repository: https://github.com/minorin2847/INTEL_SYS_DEV_2026

Hanoi, 2026

Contents

Summary **4**

1 Introduction **5**

1.1 Why sequences are different . . . 5

1.2 Networks as compositions of functions . . . 5

1.3 Questions and requirements . . . 5

1.4 How the work is organised . . . 6

2 Memory and the Five Elementary Functions **6**

2.1 What a network that reads sequences needs . . . 6

2.2 Why these five and not others . . . 7

2.3 Derivatives decide how well a gradient passes . . . 8

3 Structure of the 5-Layer RNN **8**

3.1 One cell: three elementary functions . . . 8

3.2 Stacking five layers . . . 9

3.3 Parameters . . . 11

3.4 Why depth and length hurt: the gradient . . . 12

4 Structure of the 5-Layer LSTM **13**

4.1 The idea: edit the memory, do not rewrite it . . . 13

4.2 Why each piece is there . . . 14

4.3 Stacking and composition . . . 15

4.4 Parameters . . . 15

4.5 Gradient flow and the forget-gate bias . . . 15

4.6 The two networks side by side . . . 17

5 Datasets and Preprocessing **17**

5.1 Choosing the data . . . 17

5.2 EC: Amazon reviews of unlocked mobile phones . . . 18

5.3 ST: S&P 500 daily stocks . . . 22

6 Implementation in PyTorch and Keras **26**

6.1 The same network written twice . . . 26
6.2 Weight names and the numerical check . . . 27
6.3 The same starting point in both frameworks . . . 27

7 Experimental Protocol 28
7.1 One recipe for all eight runs . . . 28
7.2 Baselines: what a network has to beat . . . 29
7.3 Metrics . . . 29
7.4 Machine . . . 30

8 Results: Review Sentiment (EC) 30
8.1 The scores . . . 30
8.2 Learning curves: overfitting after two or three epochs . . . 31
8.3 Where do the errors go? . . . 31
8.4 How confident are the models? . . . 32
8.5 Does the LSTM read long reviews better? . . . 33
8.6 Why does a bag of words win? . . . 34
8.7 Reading examples . . . 34

9 Results: Stock Volatility (ST) 35
9.1 Baselines first . . . 35
9.2 The networks beat every baseline, by a small margin . . . 36
9.3 The LSTM is only marginally better than the RNN . . . 36
9.4 Learning curves and the regime shift . . . 37
9.5 What the forecasts look like . . . 37
9.6 Where do the networks help? Calm against turbulent markets . . . 38

10 Comparison: RNN against LSTM 39

11 Discussion 41
11.1 What the experiments support . . . 41
11.2 What surprised us . . . 41
11.3 Threats to validity . . . 41

12 Conclusion and Future Work 42

13 Source Code Repository **42**

Summary

Recurrent networks are the textbook answer to data that arrives in order, such as the words of a review or the daily prices of a stock, and the LSTM is the textbook improvement of the plain RNN. This report asks whether that improvement can actually be seen, and where. To answer it, we write a **5-layer recurrent neural network (RNN)** and a **5-layer long short-term memory network (LSTM)** as compositions of only five elementary functions (matrix product, addition, tanh, sigmoid and element-wise product), show by computation that PyTorch and Keras evaluate exactly these compositions, and then train both networks in both frameworks on two Kaggle datasets of about 100 MB each: **EC**, 148,124 Amazon reviews of unlocked phones that must be classified as positive or negative, and **ST**, fifteen years of daily prices of 145 S&P 500 stocks from which the size of the next day's move must be forecast. Two datasets, two architectures and two frameworks give eight runs, which share data, protocol, starting weights and random seed. A run is named `<dataset>-<framework>-<model>`, for example `EC-PT-LSTM` for the PyTorch LSTM on the reviews.

Table 1. RNN against LSTM on the test sets. Score: macro F1 (EC) and out-of-sample R^2 against the training mean (ST); Δ = LSTM − RNN; s/epoch = median seconds per training epoch on one CPU.

Data	Framework	Score	RNN	LSTM	Δ	Params RNN	Params LSTM	s/epoch R / L	Time ratio
EC	Keras	F1 macro	0.9084	0.9161	+0.0076	1,321,345	1,445,185	43 / 198	4.6×
EC	PyTorch	F1 macro	0.8999	0.9176	+0.0177	1,321,665	1,446,465	42 / 93	2.2×
ST	Keras	R^2_{oos}	0.0884	0.0920	+0.0036	9,537	38,049	51 / 138	2.7×
ST	PyTorch	R^2_{oos}	0.0871	0.0965	+0.0094	9,697	38,689	46 / 70	1.5×

Key findings

1. **The libraries compute the formulas we write.** A NumPy implementation written directly from the equations agrees with PyTorch and with Keras to within 10^{-7} for both cells, and the hand-counted numbers of parameters match the frameworks' counts exactly (PyTorch stores one extra bias vector per layer, which we account for).
2. **The gradient explains why the LSTM exists, and why its starting point matters.** In a freshly initialised 5-layer RNN, the first step of the first layer receives only $8.7 \cdot 10^{-11}$ of the gradient found at the loss after 50 steps. An LSTM whose forget gate starts at bias 0 is just as bad ($9.4 \cdot 10^{-11}$); with forget bias 1 it receives $4.5 \cdot 10^{-4}$, about five million times more.
3. **EC: the LSTM beats the RNN in both frameworks, and more on long reviews.** Macro F1 rises by 0.8 points in Keras and 1.8 points in PyTorch, the F1 of the negative class by 0.9 and 2.7 points, and on reviews longer than 100 words the accuracy gain reaches 1.3 and 1.8 points. A bag-of-words baseline (TF-IDF with logistic regression, macro F1 0.930) nevertheless beats all four networks, because the sentiment of these reviews sits mostly in single words.
4. **ST: both networks roughly double the classical estimate, and the LSTM adds only a little.** The out-of-sample R^2 is 0.087–0.097 against 0.044 for the 20-day average of recent moves, and the LSTM's lead over the RNN is $+0.004$ (Keras) and $+0.009$ (PyTorch), because the useful memory of volatility lasts only a few weeks. The lead is largest in turbulent markets.
5. **The price.** The LSTM has four times the recurrent parameters and needs 1.5 to 4.6 times as long per epoch on a CPU. The two frameworks rank the models identically; their scores for the same architecture differ by at most 0.016 on the EC metrics and 0.005 in R^2 on ST.

1 Introduction

1.1 Why sequences are different

Most of the data we want to learn from has an order. The sentence "the phone is not good" means the opposite of "the phone is good, not bad" although the two contain nearly the same words, and a 5 % price drop means something different after a calm month than after a crash. A model that looks at each word or each day separately throws this order away. A *recurrent neural network* keeps it: it reads the sequence one step at a time and carries a small vector, the *hidden state*, from each step to the next, so that every new input is interpreted in the light of everything read before. The same network can therefore read a review of ten words and one of a hundred, or a window of 60 trading days.

Stacking several recurrent layers lets the upper layers work on summaries that the lower layers produce, but depth and length bring a problem that the whole second half of this assignment turns on. To learn from a word at the start of a review, the training signal must travel backwards through every step in between, and in a plain RNN it is multiplied by a number smaller than one at each of them, so it shrinks to almost nothing (the *vanishing gradient* [2, 3]). The *long short-term memory* network (LSTM) [1] was designed to remove exactly this obstacle: it adds a second state, the cell, which is changed by addition and controlled by learned gates, so that a signal can travel back many steps without fading.

1.2 Networks as compositions of functions

The lecturer's tutorial on deep learning asks us to see a neural network as *a structured composition of elementary functions*, and to see every new architecture as *an old architecture plus a mechanism that removes one of its limitations*. We follow both ideas literally. The RNN is a composition of three elementary functions and the LSTM of five; the LSTM differs from the RNN by one mechanism, the gated additive cell, which answers one limitation, the vanishing gradient. Writing the networks this way has a practical advantage: every number inside them can be reproduced by hand, and the claim "the framework computes the function we wrote" becomes something we can verify instead of something we have to believe.

1.3 Questions and requirements

The brief of the assignment has seven parts: (R1) two datasets of moderate size, one of user comments from e-commerce and one stock dataset from Kaggle; (R2) the structure of a 5-layer RNN as a composition of elementary functions, with explanation; (R3) the same for the 5-layer LSTM; (R4) code in PyTorch and Keras; (R5) running and evaluating the models; (R6) comparing the LSTM with the RNN on both datasets; (R7) notebooks and a report. We turn the comparison into four concrete questions:

Q1. Do PyTorch and Keras really evaluate the compositions that we write down (checked in §6)?

Q2. Does the vanishing-gradient argument hold numerically, and how much does the LSTM's cell path help at the start of training (§3, §4)?

Q3. On a task in which information is spread over many steps (reviews), does the LSTM beat the RNN, and does its advantage grow with the length of the sequence (§8)?

Q4. On a task whose useful memory is short (volatility of stocks), does the extra machinery still pay off (§9)?

Table 2. Where each requirement of the brief is answered.

#	Requirement	Sections and evidence
R1	Two datasets	§5: selection, size, cleaning, exploratory figures
R2	5-layer RNN structure	§2, §3: functions, one cell, unrolled grid, numeric example, parameters, gradient map
R3	5-layer LSTM structure	§4: gates, cell path, parameters, gradient flow, forget bias
R4	PyTorch and Keras code	§6: listings, weight mapping, numerical verification
R5	Run and evaluate	§7–§9: protocol, baselines, metrics, results, learning curves, error analysis
R6	Compare LSTM and RNN	§10: accuracy, parameters, time, behaviour; §11: discussion
R7	Notebooks and report	this report; the three notebooks and the repository of §13

1.4 How the work is organised

The work is in three Jupyter notebooks, and every number in this report is copied from their executed outputs: the concepts notebook (`00_rnn_lstm_concepts.ipynb`) derives both networks and produces the figures of §2–§4 and the checks of §6; the review notebook (`ec_reviews.ipynb`) and the stock notebook (`st_stocks.ipynb`) produce the figures and numbers of §5 and §8–§9. The report first builds the theory (§2–§4), then describes the data (§5), the implementation (§6) and the protocol (§7), reports the experiments (§8–§9), and ends with the comparison, a discussion, the conclusion and the repository.

2 Memory and the Five Elementary Functions

2.1 What a network that reads sequences needs

Suppose we want a function that reads a sequence of numbers and produces a decision, and we ask which operations it cannot do without. It must weigh every input by its importance, merge what it reads now with what it remembers, make its decisions non-linear so that several layers are more than one, and control how much of each signal passes. Five operations are enough for all of this, and they are the only ones used by both networks of the assignment (Table 3). Vectors are row vectors, so a layer is written xW.

Table 3. The five elementary functions, with the tiny examples computed in the concepts notebook.

Function	Formula	Example	Role
matrix product	$(xW)_j = \sum_i x_i W_{ij}$	$(1,2)\begin{pmatrix}1 & 0\\ 3 & 1\end{pmatrix} = (7,2)$	every unit sees every input, each with its own weight
addition	$a + b$	$(7,2) + (0.5,-1) = (7.5,1)$	merges the new input with the memory
tanh	$\frac{e^z - e^{-z}}{e^z + e^{-z}} \in (-1,1)$	$\tanh(0.5) = 0.4621$	non-linearity of the RNN and of the LSTM candidate
sigmoid	$\sigma(z) = \frac{1}{1+e^{-z}} \in (0,1)$	$\sigma(2) = 0.8808$	gate: the fraction of a signal let through
element-wise product	$(a \odot b)_i = a_i b_i$	$(0.9, 0.1) \odot (4,4) = (3.6, 0.4)$	applies a gate to each unit separately

2.2 Why these five and not others

Each choice is easiest to understand as the answer to "what goes wrong without it".

Matrix product and addition. Each hidden unit has to look at *every* input value and give each its own importance, its weight: unit j computes $\sum_i x_i W_{ij}$, and the matrix product does this for all units at once. The numbers in W are what training adjusts. The cell must also merge two sources of information, the new input and its own memory, and adding them keeps both. A useful side effect, which will matter for the LSTM, is that the derivative of a sum is 1, so a gradient passes through an addition unchanged.

A non-linearity. Without one, depth is useless: two linear layers collapse into a single one, $(xW_1)W_2 = x(W_1W_2)$, so five layers without a non-linearity would equal one matrix and could only draw straight decision boundaries. The non-linearity is what allows layers to build on each other.

tanh **rather than ReLU in a recurrence.** In a recurrent network the *same* weights are applied at every step. ReLU has no upper limit, so if the recurrent weights scale the state by 1.1 per step it has grown by $1.1^{50} \approx 117$ after 50 steps. tanh squeezes every step back into $(-1,1)$ and, being centred on zero, lets states be positive or negative. The right panel of Figure 1 shows this experiment: the tanh state settles at 0.63 while the ReLU state grows to 117.

Sigmoid and the element-wise product for gates. A gate must be a *fraction* between "block everything" (0) and "let everything through" (1), which is exactly the range of the sigmoid; tanh cannot be a gate because "let through -30 %" means nothing. A gate also has to decide for each unit separately (keep unit 1, forget unit 2), so gate and content are multiplied position by position, not through a full matrix: $(0.9, 0.1) \odot (4,4) = (3.6, 0.4)$.

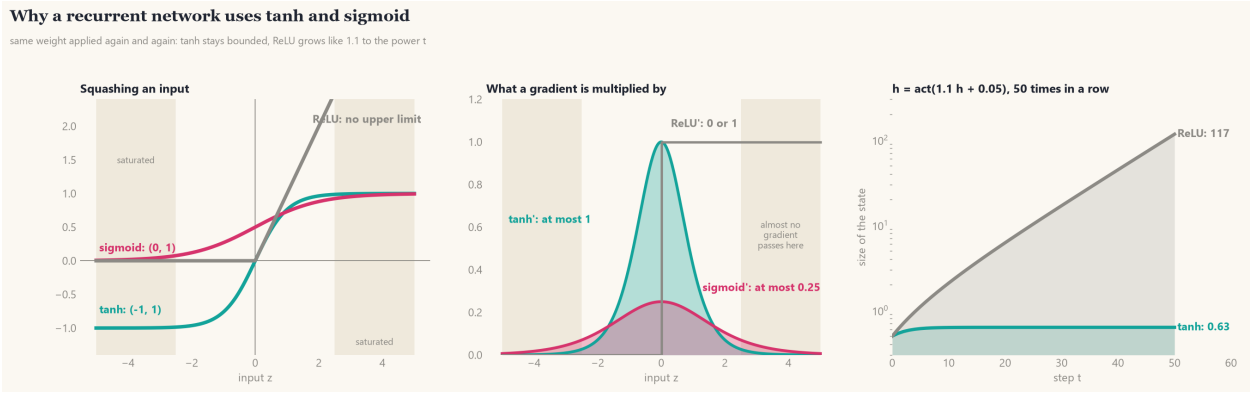


Figure 1. The activation functions (left), their derivatives (middle) and the same weight applied 50 times (right). Sigmoid and tanh have the same S shape in different ranges, both derivatives fall to nearly zero beyond about ± 2.5 (shaded), and the tanh state stays bounded while the ReLU state explodes.

2.3 Derivatives decide how well a gradient passes

Training sends a gradient backwards through every function that was applied, and the derivative of each function is the factor by which the gradient is multiplied on the way. For the two squashing functions $\tanh'(z) = 1 - \tanh^2 z \le 1$ and $\sigma'(z) = \sigma(z)(1-\sigma(z)) \le 0.25$. Both are largest at $z = 0$ and nearly zero where the unit is *saturated*: at $z = 3$, $\tanh' = 1 - 0.9951^2 = 0.0099$, so a saturated unit passes on one percent of the gradient it receives. A sigmoid unit passes at most a quarter of the gradient, a tanh unit at most all of it. This is the first hint of the problem of §3.4: whatever the weights do, the squashing functions only ever *reduce* the signal.

3 Structure of the 5-Layer RNN

3.1 One cell: three elementary functions

When we read "the phone is not good", the meaning of the last word depends on the words before it, and a network that treats each word separately cannot know that "good" was negated. A recurrent network solves this by carrying a hidden state $h_t \in \mathbb{R}^H$, a vector that summarises everything read so far. At step t the cell mixes the new input $x_t \in \mathbb{R}^F$ with its previous state:

$$h_t = \tanh\big(\underbrace{x_t W_x}_{\text{matmul}} + \underbrace{h_{t-1} W_h}_{\text{matmul}} + b\big), \qquad h_0 = 0, \qquad \text{i.e.} \quad h_t = \tanh \circ \mathrm{add}(x_t W_x,\ h_{t-1} W_h,\ b). \tag{1}$$

The formula looks like this for a reason. $x_t W_x$ is *what is new* at step t and $h_{t-1} W_h$ is *what was known before*; adding them makes the new state depend on both, and tanh keeps it bounded, so h_t is a running summary of $x_1, \dots, x_t$. The starting state is $h_0 = 0$ because nothing has been read yet.

A worked example. Take $F = H = 2$ and the weights of the concepts notebook. The first input $x_1 = (1, 0)$ meets the empty memory, so only the input contributes: the pre-activation is $a = (0.5, -0.2)$ and $h_1 = \tanh(a) = (0.4621, -0.1974)$. At $t = 2$ the cell adds the memory of step 1, $h_1 W_h$, which shifts the pre-activation to $(1.1357, 0.2756)$ and gives $h_2 = (0.8130, 0.2689)$. At $t = 3$ the input $(0, -1)$ pushes the state to $(-0.6484, 0.2966)$. Every value stays strictly between -1 and 1, and each state depends on all earlier inputs through the previous state.

Why the same weights at every step. The rule for reading a word should not depend on where the word is: "not good" means the same at word 3 and at word 80, and a 2 % price drop the same on any day. Sharing the weights also fixes the size of the network, since one W_h serves sequences of any length; separate weights for every step would multiply the number of parameters by T and could not read a sequence longer than the training examples. The price is the nesting that appears when the recurrence is unrolled:

$$h_3 = \tanh\big(x_3 W_x + \tanh\big(x_2 W_x + \tanh(x_1 W_x + b)\, W_h + b\big) W_h + b\big). \tag{2}$$

The first input reaches h_3 only through two copies of W_h and tanh, which is the origin of the gradient problem.

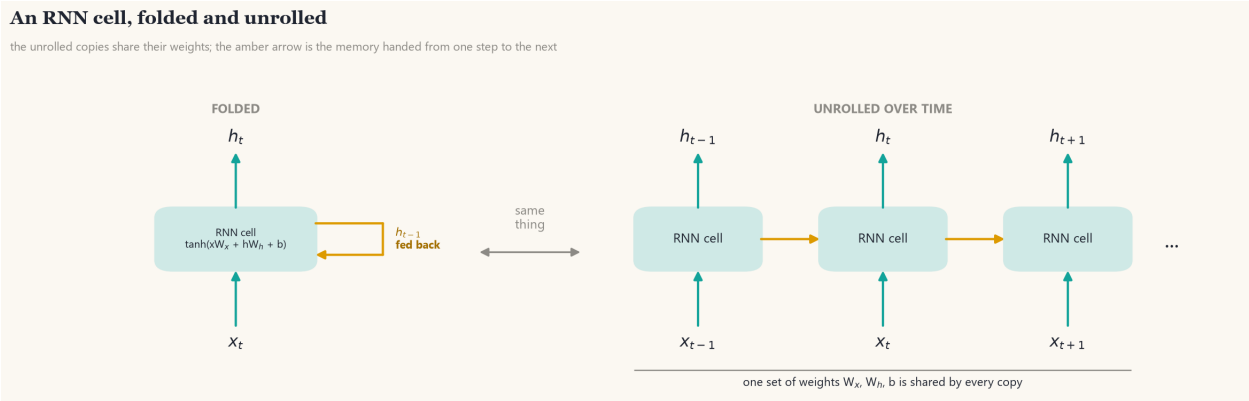


Figure 2. One RNN cell folded (left: the output is fed back into the box) and unrolled (right: the same box, with the same weights, copied once per step). The unrolled picture is the one the gradient follows in training.

3.2 Stacking five layers

One layer turns a sequence of inputs into a sequence of states, and nothing prevents us from feeding that sequence to a second layer. Layer 1 reads the input sequence and layer l reads the hidden states of layer $l-1$:

$$h_t^{(1)} = \tanh\big(x_t W_x^{(1)} + h_{t-1}^{(1)} W_h^{(1)} + b^{(1)}\big), \qquad h_t^{(l)} = \tanh\big(h_t^{(l-1)} W_x^{(l)} + h_{t-1}^{(l)} W_h^{(l)} + b^{(l)}\big),\ l = 2, \dots, 5. \tag{3}$$

The prediction uses the top layer at the last step, $\hat{y} = g\big(h_T^{(5)} W_y + b_y\big)$. If $f^{(l)}$ denotes the map from a sequence to the sequence of layer-l states, the whole network is a single composition,

$$\hat{y} = g \circ \text{last} \circ f^{(5)} \circ f^{(4)} \circ f^{(3)} \circ f^{(2)} \circ f^{(1)}\,(x_1, \dots, x_T), \tag{4}$$

in which every $f^{(l)}$ is itself T repetitions of matmul $\to$ add $\to$ tanh. Figure 3 draws the resulting grid: information moves up through the layers at the same step and right through time within a layer, and the prediction reads only the ringed state in the top-right corner.

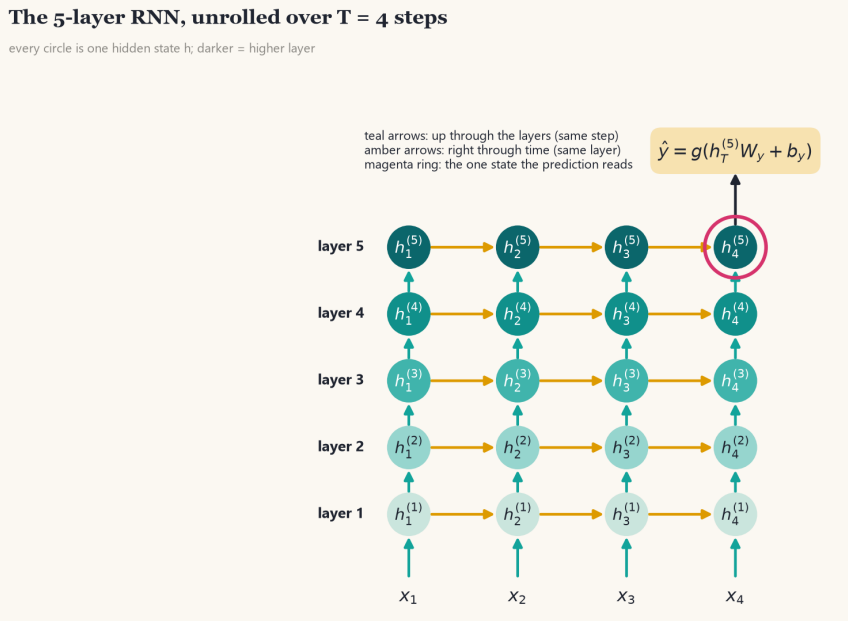


Figure 3. The 5-layer RNN unrolled over $T = 4$ steps. Darker circles belong to higher layers, teal arrows carry a state up to the next layer, amber arrows carry it to the next step, and the magenta ring marks the only state that the prediction reads.

Why stack layers. Each layer re-reads the sequence produced by the layer below, so higher layers work on more abstract summaries. On reviews, layer 1 can react to single words ("good"), a middle layer to short phrases ("not good"), and the top layer to the overall verdict; on stocks, layer 1 sees one day's move and higher layers see patterns over weeks, such as a calm or a turbulent period. One layer could in theory do all of this, but with many more units; depth reuses the features already built below.

Why only the last step of the top layer. By the recurrence, $h_T^{(5)}$ depends on every input, so it already summarises the sequence, and both of our tasks need exactly one answer per sequence. The output function g is the sigmoid for sentiment (a probability, trained with cross-entropy) and the identity for volatility (a real number, trained with squared error; squashing it would cap the forecast).

Shapes. Following the tensors through the two dataset models makes the construction concrete (Table 4): the hidden size stays H in every layer and only layer 5 drops the time axis.

Table 4. Tensor shapes for a batch of 256 sequences. EC uses a word embedding before layer 1.

Stage	EC (reviews)	ST (stocks)
input	$(256, 100)$ word ids	$(256, 60, 4)$ features
embedding $20{,}000 \times 64$	$(256, 100, 64)$	–
layers 1–4 (all steps)	$(256, 100, 64)$	$(256, 60, 32)$
layer 5, last step $h_T^{(5)}$	$(256, 64)$	$(256, 32)$
head	$(256, 1)$, sigmoid	$(256, 1)$, identity

What the five layers hold. Drawing the states of every layer for the same input shows the stack at work (Figure 4). In layer 1 the dots change from step to step, because the layer sees the raw

input and the input changes. In layers 4 and 5 most units keep nearly the same value over all six steps: these layers hold summaries of what the layers below have already processed, so they move more slowly. With random, untrained weights this smoothing is only a tendency, which a trained network would use to form its features.

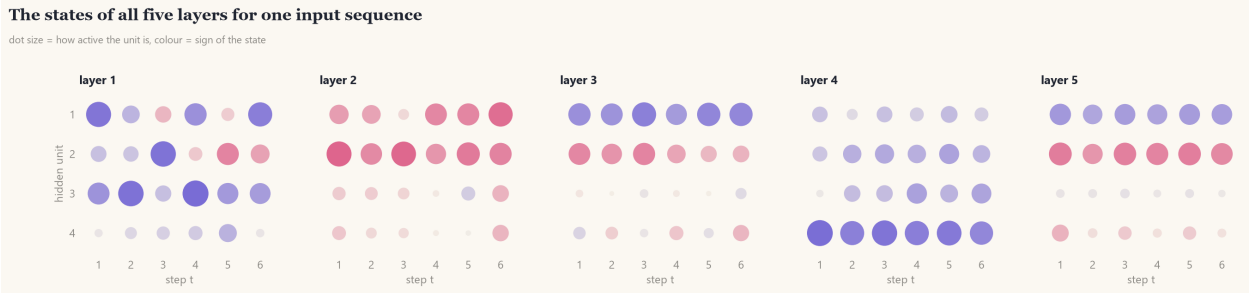


Figure 4. The hidden states of the five layers for one input sequence (rows = units, columns = steps, dot size = how strongly the unit is active, colour = sign). The upper layers change more slowly than the lower ones.

3.3 Parameters

The parameters are the numbers that training adjusts, so counting them tells us how large the model is. Layer 1 maps $F \to H$ and layers 2–5 map $H \to H$:

$$N_{\text{RNN}} = \underbrace{FH + H^2 + H}_{\text{layer 1}} + \underbrace{(L-1)(H^2 + H^2 + H)}_{\text{layers 2–5}} + \underbrace{H + 1}_{\text{head}}. \tag{5}$$

W_x has $F \times H$ numbers because each of the H units has one weight per input; above layer 1 the inputs are the H states of the layer below, so the matrix is $H \times H$. W_h has $H \times H$ numbers because each unit looks at the previous state of all H units of its layer, itself included. The bias adds one number per unit, and the head maps H numbers to one output. The count does not depend on T: a review of 10 words and one of 100 words use the same weights. For $F = 3$, $H = 4$ it is $32 + 4 \cdot 36 + 5 = 181$.

PyTorch keeps *two* bias vectors per layer (`bias_ih` and `bias_hh`, a convention inherited from the cuDNN library). Only their sum enters the formula, so the function is the same, but the count is larger by $L \cdot H$. Table 5 compares the formula with the frameworks for the toy network and for the dataset models; the dataset notebooks assert the Keras count before training, so a model that does not match the formula cannot run.

Table 5. Parameters: hand formula (one bias per layer, as Keras) against the framework counts.

Model	Formula	Keras	PyTorch	PyTorch − formula
toy RNN ($F = 3, H = 4$)	$32 + 4 \cdot 36 + 5 = 181$	181	201	$L \cdot H = 20$
toy LSTM ($F = 3, H = 4$)	$4 \cdot 32 + 16 \cdot 36 + 5 = 709$	709	789	$4LH = 80$
`EC-*-RNN` (incl. embedding)	1,321,345	1,321,345	1,321,665	320
`EC-*-LSTM` (incl. embedding)	1,445,185	1,445,185	1,446,465	1,280
`ST-*-RNN` ($F = 4, H = 32$)	9,537	9,537	9,697	160
`ST-*-LSTM`	38,049	38,049	38,689	640

3.4 Why depth and length hurt: the gradient

Training a network means asking, for every weight, "if I made this weight a little larger, would the loss go up or down, and by how much?". The answer is the *gradient* of the loss with respect to the weight, computed by the chain rule from the loss backwards through every function that was applied. For the weights that act on the first words of a review, that chain is long. With $h_t = \tanh(a_t)$ and $a_t = x_t W_x + h_{t-1} W_h + b$, going back one step in time multiplies the gradient by a matrix, and going back $T - t$ steps by $T - t$ such matrices:

$$\frac{\partial h_t}{\partial h_{t-1}} = \underbrace{\operatorname{diag}(1 - h_t^2)}_{\tanh'} W_h^\top, \qquad \frac{\partial h_T}{\partial h_t} = \prod_{k=t+1}^{T} \operatorname{diag}(1 - h_k^2)\, W_h^\top, \tag{6}$$

and going down one layer multiplies by a similar factor with W_x in place of W_h, so travelling from layer 5 to layer 1 adds four more factors.

Why it vanishes, and why it matters. Every factor contains $1 - h^2 \le 1$, and with typical starting weights each step shrinks the signal a little. In a scalar picture, if each step multiplies the gradient by 0.6, then after 50 steps $0.6^{50} \approx 8 \cdot 10^{-12}$ is left (*vanishing*); if each step multiplies by 1.2 it becomes $1.2^{50} \approx 9{,}100$ (*exploding*). The update of a weight is proportional to its gradient, so with 10^{-11} the weights that read the first words of a review learn nothing, and the RNN effectively learns from the last few steps even when the decisive word came early. Explosion can be handled by *gradient clipping* (rescaling the gradient when its length exceeds a limit; we clip at norm 1 in all runs). Vanishing cannot be clipped away, because making a tiny gradient larger would amplify noise as much as signal. It needs a different cell, which is the reason the LSTM exists.

Measuring it. We built a freshly initialised 5-layer RNN ($T = 50$, $H = 32$), put the loss on the top layer at the last step, and recorded the size of the gradient at every layer and every step (Figure 5). Looking at the map, the gradient is strongest at the last steps, close to the loss, and falls off steadily towards the left. The first step of layer 1 receives only $8.7 \cdot 10^{-11}$ of the gradient at the top-right corner, a factor of about 0.62 per step, close to the scalar example. All five layers behave alike because each repeats the same kind of shrinking, so a network trained with such gradients could not learn that a word at step 1 matters for the answer at step 50.

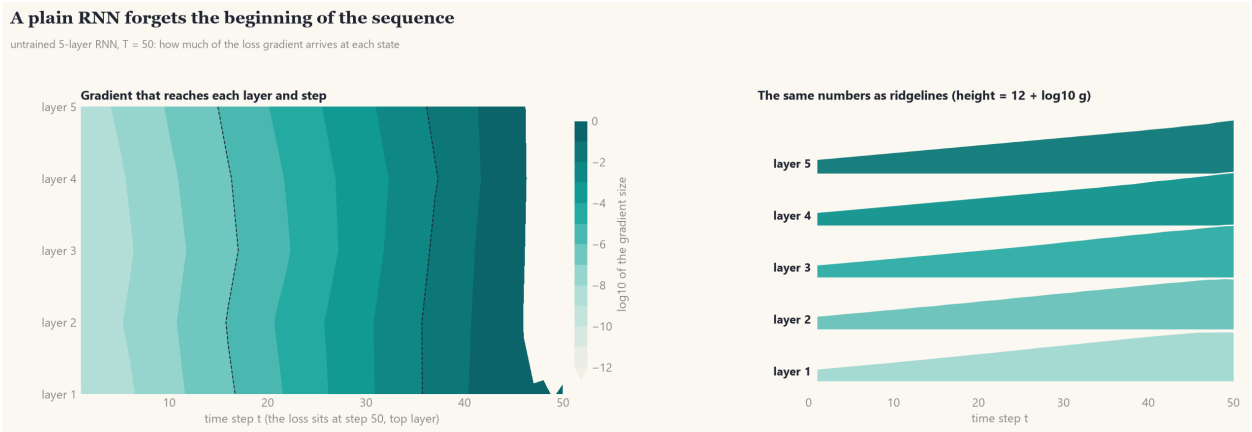


Figure 5. Size of the gradient at every layer and step of a fresh 5-layer RNN (darker = more gradient, logarithmic scale; left) and the same numbers as one ridgeline per layer (right). Step 1 of layer 1 receives $8.7 \cdot 10^{-11}$ of the gradient at the loss.

4 Structure of the 5-Layer LSTM

4.1 The idea: edit the memory, do not rewrite it

The previous section found the problem: the memory of an RNN is *rewritten* at every step, because the whole state passes through a matrix and a tanh, so information and gradient both fade. The idea of the LSTM is to give the network a second state, the **cell** c_t, that is not rewritten but only *edited*: old content can be kept, new content added, and the contents shown or hidden, all under the control of learned **gates**. A gate is a number between 0 and 1 for each unit, computed with a sigmoid, that says how much of something passes. Each gate is the RNN formula with a sigmoid in place of tanh (Table 6); the forget gate was added to the original design by Gers et al. [4].

Table 6. One LSTM step. W_* act on the input x_t, U_* on the previous hidden state h_{t-1}.

Piece	Formula	Role and reason
input gate	$i_t = \sigma(x_t W_i + h_{t-1} U_i + b_i)$	how much new content to write
forget gate	$f_t = \sigma(x_t W_f + h_{t-1} U_f + b_f)$	how much of the old cell to keep
candidate	$g_t = \tanh(x_t W_g + h_{t-1} U_g + b_g)$	new content; signed, so it can push a memory up or down
output gate	$o_t = \sigma(x_t W_o + h_{t-1} U_o + b_o)$	how much of the cell to show
cell	$c_t = f_t \odot c_{t-1} + i_t \odot g_t$	memory, updated by **addition**
hidden	$h_t = o_t \odot \tanh(c_t)$	output of the step; tanh bounds the cell, which can grow past 1

As a composition, $(h_t, c_t) = \text{LSTM}(x_t, h_{t-1}, c_{t-1})$ uses matrix products, additions, sigmoid, tanh and $\odot$ and nothing else. A small example shows the cell update: with $f = 0.9$, $c_{t-1} = 2.0$, $i = 0.5$, $g = 0.6$ we get $c_t = 0.9 \cdot 2.0 + 0.5 \cdot 0.6 = 1.8 + 0.3 = 2.1$, so the cell keeps 90 % of what it had and adds half of the new content.

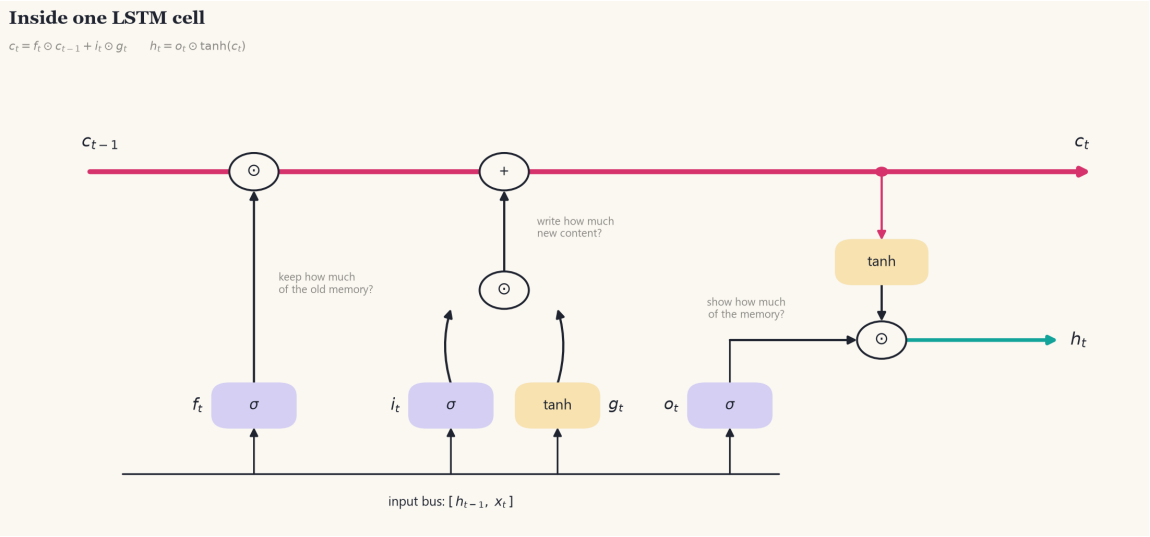


Figure 6. One LSTM cell. The top line is the cell state, the memory highway: it passes through the cell with only one multiplication (by the forget gate) and one addition (of the new content), with no weight matrix and no squashing. Everything below the line is gate machinery that reads $[h_{t-1}, x_t]$ and decides what happens on the highway.

4.2 Why each piece is there

The gates read x_t and h_{t-1}. The decision to forget or to write depends on context. After "not" the network should treat the following "good" differently, and only h_{t-1} knows that "not" came before.

The candidate uses tanh **and the gates use sigmoid.** New content can push a memory up or down, so it needs signed values in $(-1, 1)$, exactly like the RNN state; gates are fractions, as argued in §2.

The cell is updated by addition. In $c_t = f_t \odot c_{t-1} + i_t \odot g_t$ the old memory is *copied*, scaled by f_t, and not pushed through a matrix and a tanh again. This changes the gradient completely: going back one step along the cell gives

$$\frac{\partial c_t}{\partial c_{t-1}} = \text{diag}(f_t) \quad \text{(plus small indirect terms)}, \tag{7}$$

so the gradient is multiplied by the forget gate only, with no weight matrix and no squashing derivative. If $f \approx 0.97$, then $0.97^{50} \approx 0.22$ survives 50 steps, against $0.6^{50} \approx 8 \cdot 10^{-12}$ for the RNN of §3.4.

$h_t = o_t \odot \tanh(c_t)$. The cell is a sum and can grow beyond 1, and tanh brings the output back into $(-1, 1)$ for the next layer. The output gate lets the cell *hold* information without showing it until it is useful: a word read at step 5 can be released at step 90.

Nobody sets the gates by hand. Training adjusts W_f, U_f, b_f (and the others) so that $f_t \approx 1$ where memory pays off, such as an early "but" that flips the meaning of a review, and $f_t \approx 0$ where the cell should reset.

Gates in action. Figure 7 sets the gates by hand on a tiny memory task: a value $+1$ is written into the cell at step 3 and must still be readable at step 20. It is not a trained network; it only shows the cell equation at work. Writing is done by briefly opening the input gate (step 3) and reading by opening the output gate (step 20); in between the forget gate stays high. The right panel keeps the write at step 3 and varies the forget gate only, which shows why that gate is the key number. With $f = 1$ nothing is lost; with $f = 0.97$, a value a trained gate can reach, 60 % of the value (0.596) remains after 17 steps; with $f = 0.73$, the value a fresh LSTM has when the forget bias is 1, only 0.5 % (0.0047); with $f = 0.5$ (bias 0) practically nothing, $0.5^{17} \approx 8 \cdot 10^{-6}$. The architecture makes long memory *possible*, but training still has to find it.

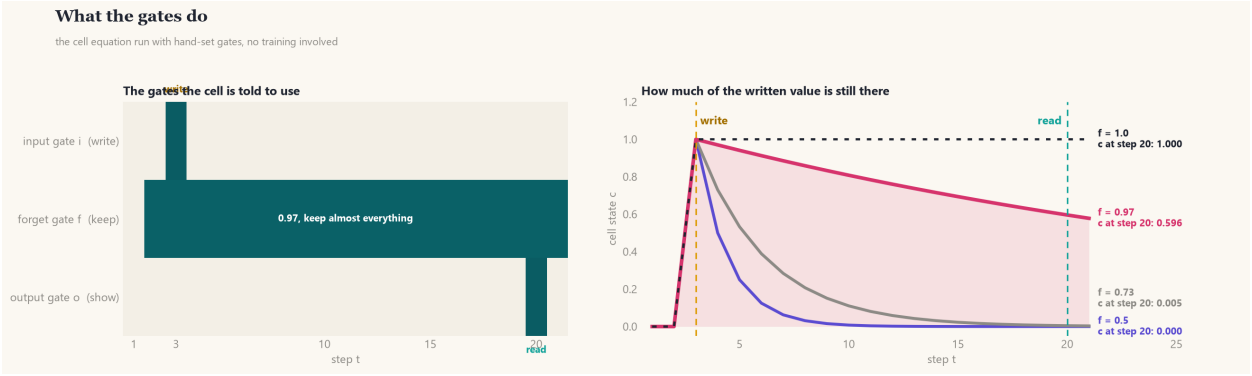


Figure 7. Hand-set gates on a memory task: the value written at step 3 is read at step 20 (left and middle), and what remains of it after 17 steps for different values of the forget gate (right).

4.3 Stacking and composition

Stacking works as for the RNN: layer l reads $h_t^{(l-1)}$, and each layer has its own cell $c_t^{(l)}$,

$$\left(h_t^{(l)}, c_t^{(l)}\right) = \mathsf{LSTM}^{(l)}\left(h_t^{(l-1)},\, h_{t-1}^{(l)},\, c_{t-1}^{(l)}\right), \qquad \hat{y} = g \circ \mathsf{last} \circ F^{(5)} \circ \cdots \circ F^{(1)}(x_1, \ldots, x_T). \tag{8}$$

Both frameworks store the four gates side by side in one matrix of width $4H$, in the order **i, f, g, o** (Keras calls the candidate "c"). All four gates read the same inputs, so concatenating their weights turns four small matrix products into one large one, which a processor computes faster; the result is then split into four blocks of H columns. The order is only a storage convention, but it must be respected when weights are copied between frameworks.

4.4 Parameters

Each layer has four copies of the RNN's weights, one per gate:

$$N_{\mathsf{LSTM}} = 4(FH + H^2 + H) + 4(L-1)(2H^2 + H) + H + 1. \tag{9}$$

For $F = 3$, $H = 4$ this is $4 \cdot 32 + 16 \cdot 36 + 5 = 709$, about four times the RNN's 181. Each of the four blocks (i, f, g, o) has exactly the shape of one RNN layer (an input matrix, a recurrent matrix and a bias), and the cell update and h_t only combine the blocks with $\odot$ and $+$, which adds no parameters; hence exactly $4\times$ the recurrent part (Table 5). On ST this is the whole model (9,537 $\rightarrow$ 38,049); on EC the 1.28 M-parameter embedding is shared and dominates, so the total grows by only 9 % (1,321,345 $\rightarrow$ 1,445,185). The RNN is deliberately *not* given four times more units to compensate, because then the comparison would change the size and the architecture at the same time.

4.5 Gradient flow and the forget-gate bias

We repeat the experiment of §3.4 for a 5-layer LSTM with two settings of the **forget-gate bias**. The bias b_f is the number added to the forget gate's pre-activation, so it sets how much the gate lets through at the beginning of training, before the weights have learned anything: $f \approx \sigma(b_f)$. Keras starts with bias 1 ($f \approx \sigma(1) = 0.73$), the choice recommended by Jozefowicz et al. [5], and PyTorch with bias 0 ($f \approx \sigma(0) = 0.5$). At the start of training the gradient that passes back through the cell is roughly f^T:

- bias 0: $\sigma(0) = 0.5$ and $0.5^{50} \approx 9 \cdot 10^{-16}$, so the cell starts out forgetting almost everything

and the gradient vanishes as badly as in the RNN;

- bias 1: $\sigma(1) = 0.73$ and $0.73^{50} \approx 1.5 \cdot 10^{-7}$, about 10^8 times larger, so training has a signal to push f towards 1 where memory pays off.

The principle is to start the network *remembering* and let it learn what to forget: if it starts out forgetting, it can never discover what was worth remembering, because no gradient reaches the early steps to tell it. Our dataset notebooks therefore set the bias to 1 in PyTorch too.

Figure 8 shows the three networks side by side. All have the same size, input and loss; only the cell type and the forget bias differ. In the RNN panel the gradient at step 1 is about ten orders of magnitude weaker than at the loss. The LSTM with bias 0 looks the same: the first step of layer 1 receives $9.4 \cdot 10^{-11}$ of the top gradient, against $8.7 \cdot 10^{-11}$ for the RNN, so the LSTM architecture on its own does *not* cure the vanishing gradient at the start of training. With bias 1 the whole map is dark, and the first step of layer 1 receives $4.5 \cdot 10^{-4}$, about five million times more than the other two. The cell path only helps if the forget gates start out open.

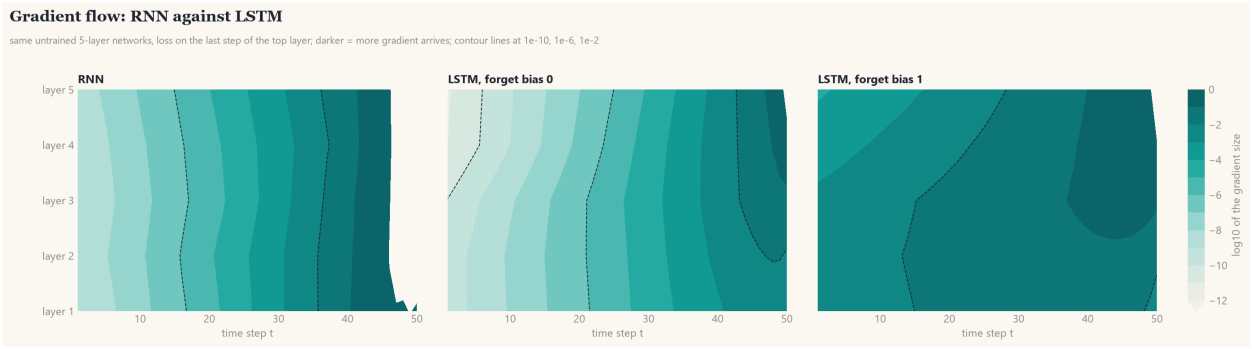


Figure 8. Gradient maps of the 5-layer RNN, the 5-layer LSTM with forget bias 0 and the 5-layer LSTM with forget bias 1 ($T = 50$, $H = 32$, loss on the top layer at the last step; contour lines at 10^{-10}, 10^{-6} and 10^{-2}). Step 1 of layer 1 receives $8.7 \cdot 10^{-11}$, $9.4 \cdot 10^{-11}$ and $4.5 \cdot 10^{-4}$ of the gradient at the loss.

The same experiment as one curve per model makes the rate of loss readable (Figure 9). For layer 1 alone, it plots how much gradient survives when the loss sits k steps later. The RNN and the LSTM with bias 0 follow the dotted line for a factor of about 0.6 per step, down to 10^{-10} after 50 steps, nearly on top of each other. The LSTM with bias 1 starts lower (at small k the signal still has to cross four layers) but then runs much flatter: between $k = 12$ and $k = 49$ it loses only about one order of magnitude, a factor of about 0.94 per step, steeper than the dotted line of 0.97 but far better than 0.6. At 49 steps the difference is between six and seven orders of magnitude. This is the mechanism the dataset experiments rely on: the LSTM can learn from information that lies many steps back, *provided* its forget gates start open and learn to stay open where it matters.

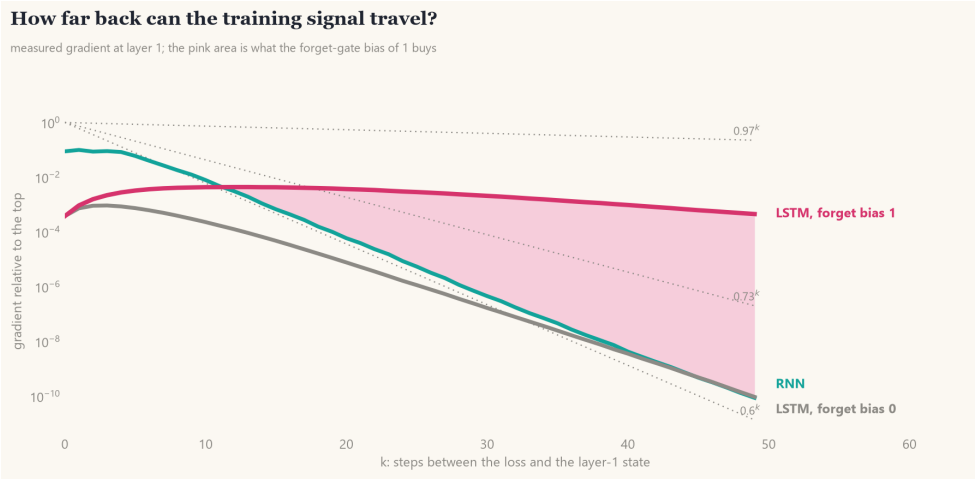


Figure 9. Gradient reaching layer 1 as a function of the distance k to the loss, for the three networks (coloured), against the scalar arithmetic f^k (dotted). The pink area is what the forget bias buys.

4.6 The two networks side by side

Table 7. 5-layer RNN and 5-layer LSTM side by side.

	5-layer RNN	5-layer LSTM
elementary functions	matmul, add, tanh	matmul, add, tanh, σ, $\odot$
state per layer	h_t	h_t and c_t
one step	$h_t = \tanh(x_t W_x + h_{t-1} W_h + b)$	three gates and a candidate, $c_t = f \odot c_{t-1} + i \odot g$, $h_t = o \odot \tanh c_t$
parameters per layer	$F_l H + H^2 + H$	$4(F_l H + H^2 + H)$
gradient back one step	$\times \operatorname{diag}(1 - h^2) W_h^\top$: shrinks	through c: $\times f_t$, kept when $f_t \approx 1$
PyTorch / Keras	`nn.RNN(num_layers=5)` / 5 × `SimpleRNN`	`nn.LSTM(num_layers=5)` / 5 × `LSTM`

The theory predicts when each should be chosen: the LSTM's advantage is the cell path, which only pays off when the answer depends on information far back in the sequence. For short sequences, or when the last few steps carry most of the signal, a plain RNN should do nearly as well at a quarter of the recurrent parameters. The two dataset experiments are designed to test exactly this prediction.

5 Datasets and Preprocessing

5.1 Choosing the data

The brief asks for user comments from e-commerce and for a Kaggle stock dataset, each of moderate size: large enough to train a 5-layer network, small enough to run on one CPU. The two datasets we chose (Table 8) also give the two architectures different things to prove. In a review the evidence is spread over many words, a complaint may come after a polite opening, and a negation may flip a phrase, so EC asks for *long* memory. Stock volatility has structure too, but, as the exploration will show, it is concentrated in the recent weeks, so ST tests whether a long memory still helps when the signal is mostly recent.

Table 8. The two datasets (both from Kaggle).

	EC	ST
Kaggle source	PromptCloudHQ/amazon-reviews-unlocked-mobile-phones	andrewmvd/sp-500-stocks
raw file	`Amazon_Unlocked_Mobile.csv`, 132 MB, 413,840 rows	`sp500_stocks.csv`, 96 MB, 1,891,536 rows
content	product, brand, price, rating 1–5, review text, helpfulness votes	date, symbol, open, high, low, close, adjusted close, volume (502 stocks, 2010–2024)
task	review sentiment: positive (4–5 stars) or negative (1–2 stars)	size of the next day's move, $\lvert r_{t+1} \rvert$
examples used	148,124 reviews	537,515 windows of 60 days, from 145 stocks
split	stratified random, 70/15/15 %	chronological, 70/15/15 % by target date

5.2 EC: Amazon reviews of unlocked mobile phones

What the file contains. The first rows show six columns: the product name and the brand (text), the price (a decimal number), the star rating (a whole number from 1 to 5), the review text (a string) and the number of helpfulness votes (a decimal, because missing values force pandas to store the column as a float). Only two columns matter for the task: the text is the input and the rating is the source of the label. Product, brand and price might let a model guess that cheap phones get worse reviews, but that would not be *reading* the review, so they are left out.

Why we drop duplicates. Two questions decide the cleaning: how many reviews have no text, and is every row really a different review? Only 70 rows have no text, which is nothing among 413,840, so they are dropped. The surprise is that **251,349 rows repeat a review text that already appeared earlier**. A phone is often sold in several variants (colours, storage sizes, carriers) with their own listings, and a shop page shows the reviews of the whole product family under every variant, so each review was collected once per variant. This matters for more than tidiness: if a review appears five times and the file is split at random, some copies fall into the training set and others into the test set, and the network is "tested" on text it has already seen, so its accuracy would measure memory, not understanding. We therefore keep one copy of each text *before* splitting. Of the 413,770 rows with text, only 162,490 (39 %) are different reviews (Figure 10, left).

Why we drop the 3-star reviews. Four stars and five stars are clearly positive and one and two stars clearly negative, but a three-star review ("it's okay", "good but…") is ambiguous: forcing it into one class would add label noise that no model can learn from. Removing the 14,366 three-star reviews leaves **148,124 labelled reviews**, still plenty for a model with over a million parameters.

Reading the rating chart. Counting the unique reviews by star rating (Figure 10, right), customers give 5 stars almost half of the time (48 %), 1 star is the second most common rating (20 %), and the middle ratings are rare (2 stars 7 %, 3 stars 9 %, 4 stars 16 %). People write a review when they are delighted or angry, not when something is merely fine. After removing the 3-star reviews, 70.1 % of the remaining reviews are positive, which has a practical consequence: a "model" that answers *positive* to every review is already right 70 % of the time, so accuracy alone would flatter

a lazy model. We therefore also report the F1 score of the negative class, the macro-averaged F1 and the ROC-AUC (§7), which punish a model that ignores the minority class.

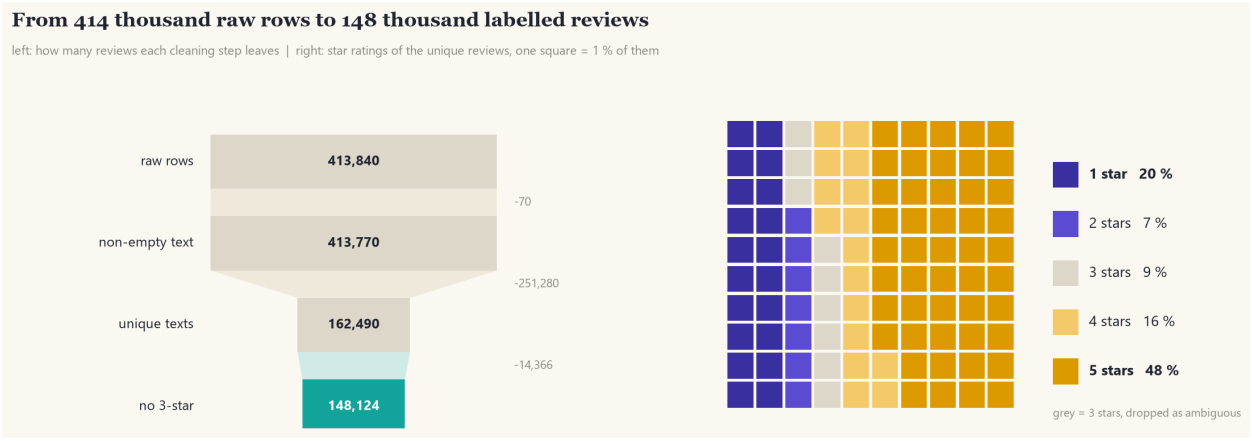


Figure 10. EC: the cleaning funnel (left) and the unique reviews by star rating, one square per percent (right). Most of the file is repetition, and the ratings are dominated by 5 stars and 1 star.

How long are the reviews? A recurrent network reads one word per step, so the number of words is the number of steps the network must run and the number of steps it must remember across: length decides both the computation time and the difficulty of the memory task. The median review has 24 words (the mean is 49, pulled up by a few reviews with thousands of words), and the medians climb as the rating gets worse: 20 words for 5-star reviews, 28 for 4-star, 30 for 1-star and 35 for 2-star reviews. Satisfied customers write "great phone, works perfectly" and move on; unhappy customers explain what went wrong and what they tried. The bars on the right of Figure 11 make the same point from the other side: among the shortest reviews (1–10 words) only 19 % are negative, among reviews of 41–100 words about 40 % are, against 30 % overall. This is an important hint for the experiment: negative evidence is spread over more words, so a model with a longer memory has a real chance to help on exactly the reviews that matter most to a shop. (Above 100 words the share drops to 34 %, but those reviews are cut at $T = 100$.)

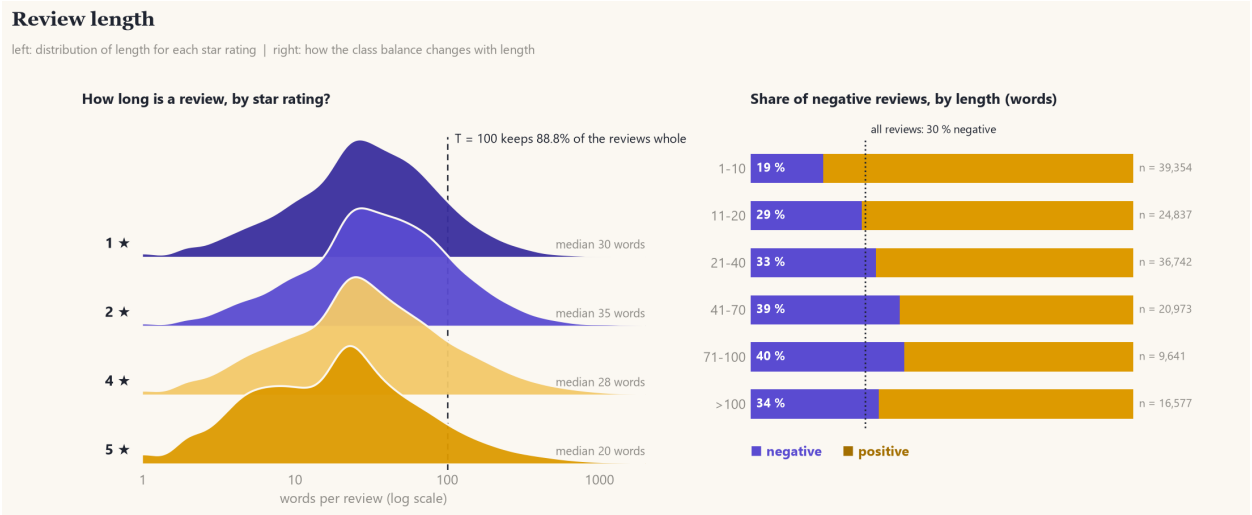


Figure 11. EC: review length by rating (left, logarithmic axis, the dashed line marks $T = 100$) and the share of negative reviews by length (right). Negative reviews are longer.

Which words give a review away? Before any neural network we can ask whether single words separate the classes. For every word we compare the share of positive reviews that contain it with the share of negative reviews that contain it (Figure 12); a word that says nothing sits on the diagonal, and the further it lies from the diagonal the better it separates the classes. A word is counted once per review, so "refund refund refund" does not outweigh the others. The words above the diagonal are what an angry customer says (*waste, worst, refund, junk, horrible, defective, stopped, return, repair, freezes, useless*), the words below it belong to satisfied customers (*excellent, love, perfect, amazing, awesome, fantastic, fits*), and even misspellings such as *excelente* are strong positive signals, so the vocabulary should be built from the data itself. One word deserves a second look: *complaints* is more typical of *positive* reviews, because of "no complaints". A method that counts words independently cannot see this; a model that reads the words in order can. It also warns us not to expect too much: so many signal words are as unambiguous as *refund* and *excellent* that a method which merely counts words will already be strong, which is exactly what the baseline of §7 tests.

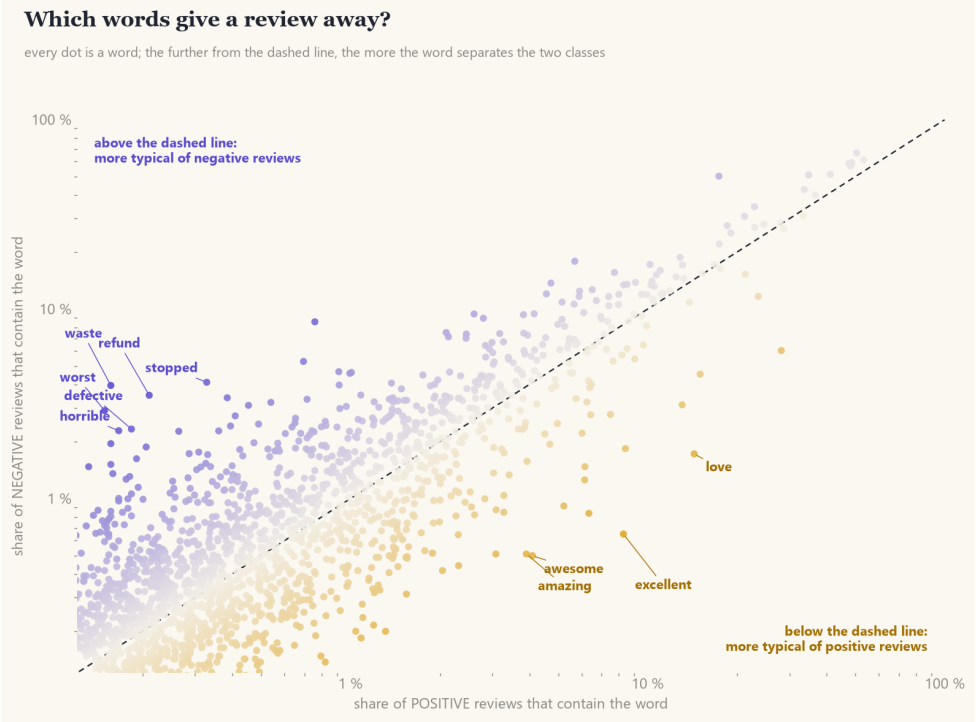


Figure 12. EC: share of positive reviews against share of negative reviews that contain each word. Words far above the diagonal are typical of complaints, words far below it of praise.

From text to numbers. A neural network can only multiply numbers, so a review first becomes a row of numbers, in four steps (Figure 13). (1) *Tokenise*: split the text into lower-case words; lower-casing makes "Great" and "great" one word and keeping apostrophes keeps "don't" in one piece. (2) *Build a vocabulary*: give each of the 20,000 most frequent words of the *training* reviews a whole number, its id (ids 2, 3, 4, …by frequency); id 1 is reserved for <unk>, which replaces every rarer word, and id 0 is the padding symbol. The vocabulary must be limited because every word costs 64 learned numbers, and words that occur once or twice give the network nothing to learn from. (3) *Encode*: replace each word by its id. (4) *Pad or truncate to* $T = 100$: the network is fed a rectangular table, so shorter reviews are filled with zeros and longer ones are cut after 100 words. The zeros go *in front* (pre-padding), because the classification is made from the final state h_T, and it is better that the last thing the network reads is a real word and not a long run of zeros to be remembered through.

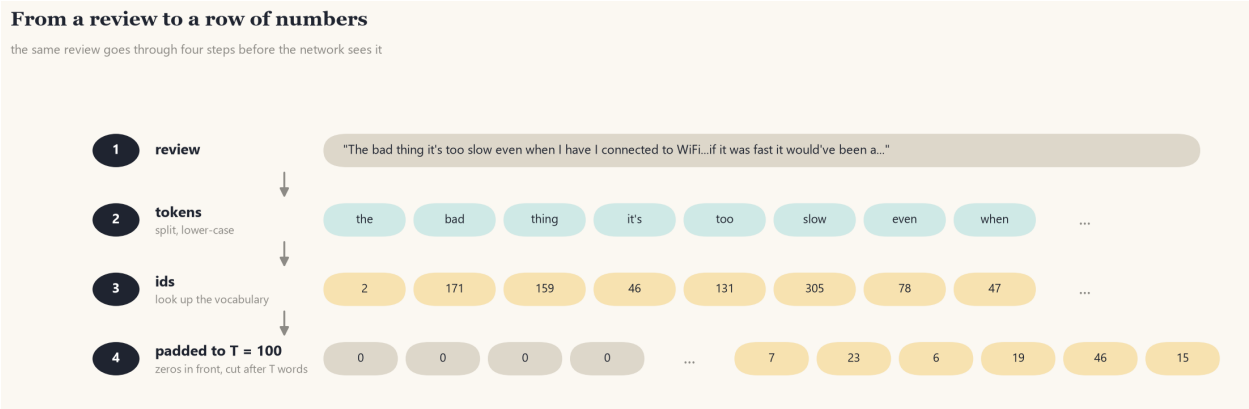


Figure 13. EC: one real review followed through the four steps (tokens, ids, padded sequence, decoded again). The ids are small for frequent words (the = 2, bad = 171) and the padded version starts with zeros and ends with the real words.

Split first, then build the vocabulary. The reviews are divided into 70 % training, 15 % validation and 15 % test reviews (103,686 / 22,219 / 22,219), stratified by label so that each part has the same share of positive reviews. Only then is the vocabulary built, from the training part alone: had it been built from all reviews, words occurring only in the test set would receive ids and the test set would have leaked into the set-up of the model; this way the model meets such words as <unk>, exactly as it would in reviews after deployment. The three parts have different jobs: the training reviews are what the network learns from, the validation reviews decide when to stop, and the test reviews are used once for the final scores. A random split is valid here because reviews are independent of one another.

How big a vocabulary, and how long a sequence? The training reviews contain 54,188 different words, but language is very uneven: a few words (*the, i, it, and, phone*) appear all the time, while 55 % of the different words (typos, model numbers, names) occur exactly once. Coverage climbs steeply at first (the 1,000 most frequent words cover 87 % of all word occurrences, the 5,000 most frequent 97 %) and then flattens: going from 10,000 to 20,000 words adds less than one percentage point (98.5 % to 99.3 %) while doubling the size of the embedding table (Figure 14). After encoding, only 0.73 % of the real tokens are <unk>. The choice $T = 100$ is a trade-off: a larger T keeps more of the long reviews but makes every training step slower (the steps of a recurrence cannot be computed in parallel) and gives the network more to remember. With $T = 100$ about 89 % (88.8 %) of the reviews fit completely.

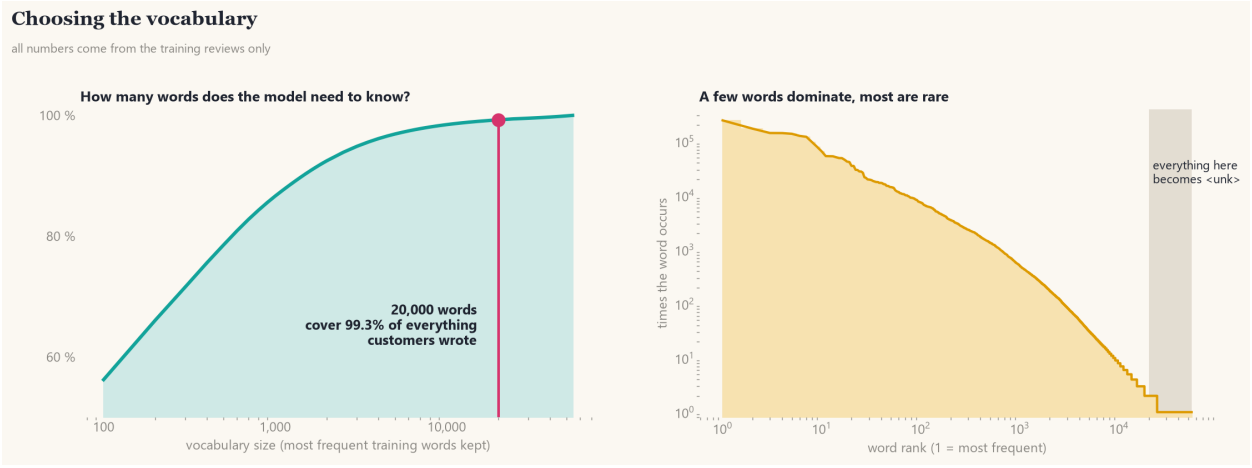


Figure 14. EC: the share of all word occurrences covered by the most frequent words (left) and the frequency of words by rank on a logarithmic scale (right). The grey zone beyond rank 20,000 holds words that occur a handful of times.

5.3 ST: S&P 500 daily stocks

What we ask, and why not the direction. It is tempting to ask a recurrent network whether a stock will go up or down tomorrow. The honest answer is that nobody can do this reliably, and the first job of the stock experiment is to choose a question that *can* be answered. A *return* is the relative price change of one day (+2 % means the price rose by two percent), and its absolute value $|r|$ ignores the sign and measures only how large the move was; averaged over time, the size of daily moves is what finance calls *volatility*. Forecasting it is a real task: risk managers use it to decide how much money to put at risk, and option prices depend on it. We therefore forecast $|r_{t+1}|$ from the last 60 trading days (about three months) of a stock.

Choosing the stocks. The file is in *long* format, one row per (date, stock) pair, with the columns Date, Symbol, Adj Close, Close, High, Low, Open and Volume. *Adj Close* is the closing price corrected for splits and dividends, so that a 2-for-1 split does not look like a 50 % crash; we use it to compute returns. A glance at the counts shows a problem: of the 1,891,536 rows, 1,273,705 (67 %) have no price, because the file covers every stock *currently* in the index and a company that joined in 2018 has empty rows for 2010–2017. Windows need an unbroken run of 60 days and we want all stocks to share one calendar so that a single chronological split works, so we keep the **145 stocks that have all 3,768 trading days** (2010-01-04 to 2024-12-20). This selection has a cost that should be stated openly: it suffers from **survivorship bias**. Stocks that left the index (often because they did badly) and stocks that joined late are missing, so our companies are those that were large enough to stay for fifteen years, and their prices grew faster than a random sample's would have. Because all four models see exactly the same data this matters little for the comparison, but it is a reason not to quote our volatility levels as facts about the market in general.

Features. For stock s on day t we build four numbers from the raw prices, each for a reason (Table 9); the target is the size of the next day's move.

Table 9. ST: the four daily features and the target.

Feature	Formula and example	Why
log return	$r_t = \ln(P_t/P_{t-1})$; price $100 \to 102$ gives $\ln 1.02 = 0.0198$	log returns add up over days, and a rise and a fall of the same size have the same magnitude; P_t is the adjusted close
high–low range	$(\text{High} - \text{Low})/\text{Close}$; high 103, low 99, close 102 gives 0.039	the span covered *during* the day, a direct measure of intraday volatility (a day that swings 4 % and closes flat was turbulent)
open–close return	$\ln(\text{Close}/\text{Open})$; open 101, close 102 gives 0.0098	the move inside the session, separate from the overnight jump
log volume	$\ln(\text{Volume})$, z-scored per stock	volume jumps on turbulent days; the logarithm tames its range and the per-stock scaling removes the size of the company
target	$y_t = \lvert r_{t+1} \rvert$; tomorrow $102 \to 99.5$ gives 0.0248	the size of tomorrow's move; known only after the day ends, so it can never leak into the features

Looking at the raw material. Three pictures (Figure 15) show what the data look like before any model sees it. On the left, the same fifteen years turn $1 into very different amounts, about $12 for GOOG but $304 for NVDA: prices wander without a fixed level and would be useless as network inputs (a network trained on the low prices of 2010 would see 2024 as an alien world), whereas returns are relative changes and have no such trend. The middle chart plots, for every size x measured in standard deviations, the share of days on which the return was at least that large. For a normal distribution the curve falls so fast that days beyond 5 standard deviations should practically never happen; in reality 0.34 % of all stock-days are that large, about 5,900 times more often than a normal distribution allows, and the kurtosis of 20 (a normal distribution has 0) says the same in one number. Because the loss is the mean of *squared* errors, these few days weigh heavily: one 15 % day contributes as much error as a hundred 1.5 % days, which is why forecasting volatility day by day is hard and why we should expect a low R^2. The raincloud on the right show that the target is skewed to the right (most days are small and a few are huge, so the mean of 1.24 % in the training period lies above the median of 0.83 %) and that the periods differ: the validation period was clearly more turbulent (mean 1.45 %, median 1.06 %) than the training period, while the test period (mean 1.27 %) is roughly like the training period. A model that learned the calm years has to forecast a stormier world in validation, which will make its validation loss look worse than its training loss even before any overfitting.

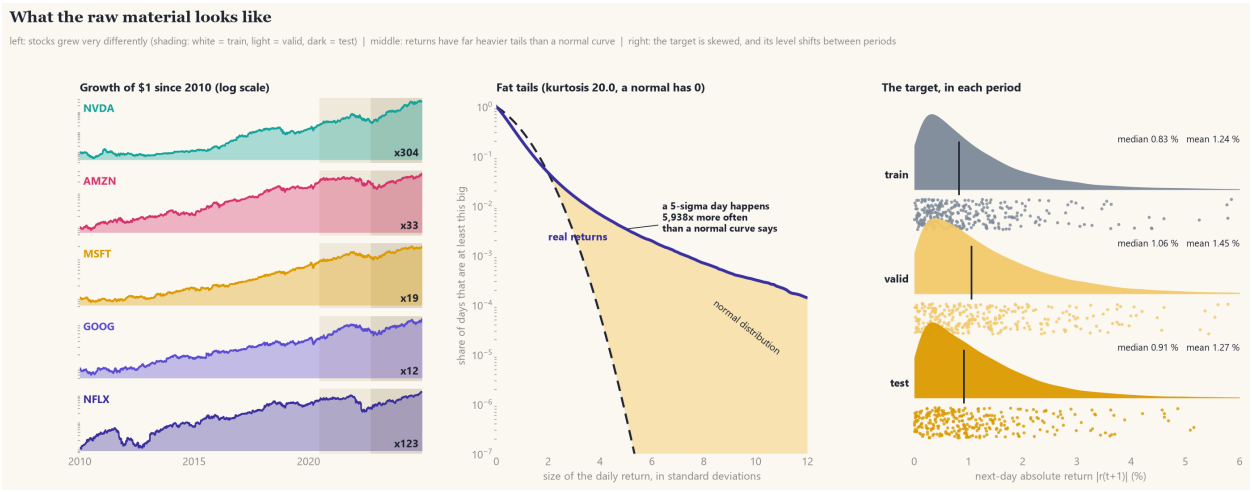


Figure 15. ST: normalised prices of five stocks with the three periods shaded (left), the share of days whose return exceeds a given number of standard deviations compared with the normal distribution (middle) and the distribution of the target in each period (right).

Volatility clustering: why $|r|$ is predictable and r is not. To measure how much a series remembers its own past we use the *autocorrelation* at lag k, $\rho_k = \mathrm{corr}(z_t, z_{t-k})$, the ordinary correlation between the series and the same series shifted by k days. If tomorrow tends to resemble today, ρ_1 is positive; if there is no relationship it is close to 0, and with a long series pure noise stays within about ± 0.03. We compute it for the signed return r_t (does a rise predict a rise?) and for its size $|r_t|$ (does a big move predict a big move?) and average over the stocks (Figure 16). The signed return has no memory: its autocorrelation is -0.040 at lag 1 and stays within the noise band up to lag 60, so yesterday's direction says nothing reliable about tomorrow's. The size of the move is the opposite: ρ_k is 0.215 at lag 1, 0.222 at lag 2, still 0.178 at five days, 0.110 at twenty days, 0.077 at forty days and 0.048 at sixty days, all clearly above the noise band. The memory of volatility is long, but it fades, and halves after about 20 days. The calendar of monthly average moves shows the same thing as runs of dark cells: August to November 2011 (the US credit downgrade and the euro crisis) averages 2.0–2.8 % per day, March and April 2020 (Covid) reach 5.9 % and 3.3 %, 2022 (rapid interest-rate rises) stays between 1.3 and 2.0 % for the whole year, and the quiet years 2013, 2014 and 2017 sit at about 1 %. Volatility is something a stock *has* for a while (*volatility clustering*, one of the best-documented "stylised facts" of financial returns [9]), which is why a recurrent network that accumulates evidence over time should be able to forecast it, and why the *length* of its memory is what the LSTM-versus-RNN comparison is about.

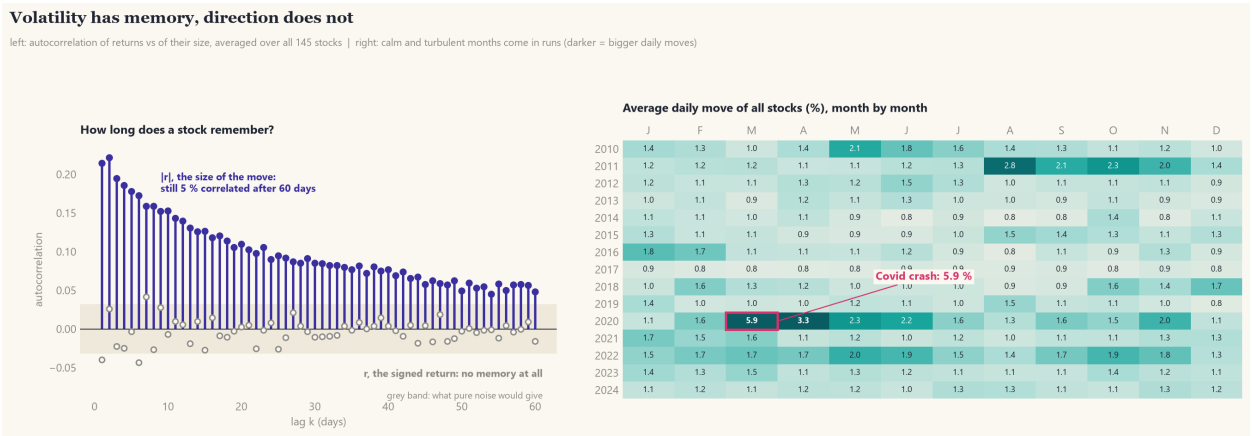


Figure 16. ST: autocorrelation of the signed return (inside the noise band) and of its size (clearly positive, decaying over weeks), and the calendar of the average daily move per month.

Scaling, windows and the chronological split. Features of very different size (a return of 0.02, a log volume of 20) confuse a network, so each feature is *z-scored*, $z = (x - \mu)/\sigma$, so that it has mean 0 and standard deviation 1 and a value of 2 means "two standard deviations above usual". The statistics μ and σ come from the **training period only**, since using the validation or test period would let information from the future into the preprocessing. The series of four features is then cut into overlapping windows: the input of one example is the features of days $t-59, \ldots, t$ and the target is $|r_{t+1}|$ (with $T = 3$, days 1–3 predict $|r_4|$ and days 2–4 predict $|r_5|$, so each day is used in many windows). A window belongs to the training, validation or test set by the date of its *target*, so no target crosses a boundary. The cut dates are 2020-06-25 and 2022-09-22: training uses 2010 to mid-2020 (373,665 windows), validation mid-2020 to autumn 2022 (81,925) and testing the last two and a quarter years (81,925). A random split of days would have been a mistake, because volatility clusters: a turbulent day in March 2020 has neighbours in training and in test that look almost the same, and the model would be rewarded for remembering the neighbours. Splitting by time mimics real use, a forecast made from the past about a future not yet seen, and the price of this honesty is that the three periods differ, as we saw. Neighbouring windows of the same stock share 59 of their 60 days, so the 373,665 training examples carry less independent information than their number suggests; this is one more reason why the networks overfit quickly in §9.

What one example looks like. Figure 17 shows a window as a 60×4 table of z-scores (a barcode: each column a day, each row a feature) together with the number to predict, and how the features relate to the target. The target is clearly related to the *range and volume* features and not to the signed ones: the high–low range has a correlation of 0.41 with tomorrow's move, the absolute return of today 0.28 and the log volume 0.21, while the signed log return and the open–close return have correlations around -0.06, that is, nothing. The log return and the open–close return are almost the same thing (correlation 0.82), so one of them is partly redundant. All of this fits the clustering picture: how wild today was (range, volume, absolute return) is informative about tomorrow, which way the price went is not. A correlation of 0.41 means that this single feature explains about 17 % of the variance of tomorrow's move ($0.41^2 = 0.17$), a number to remember when we judge the R^2 of the networks.

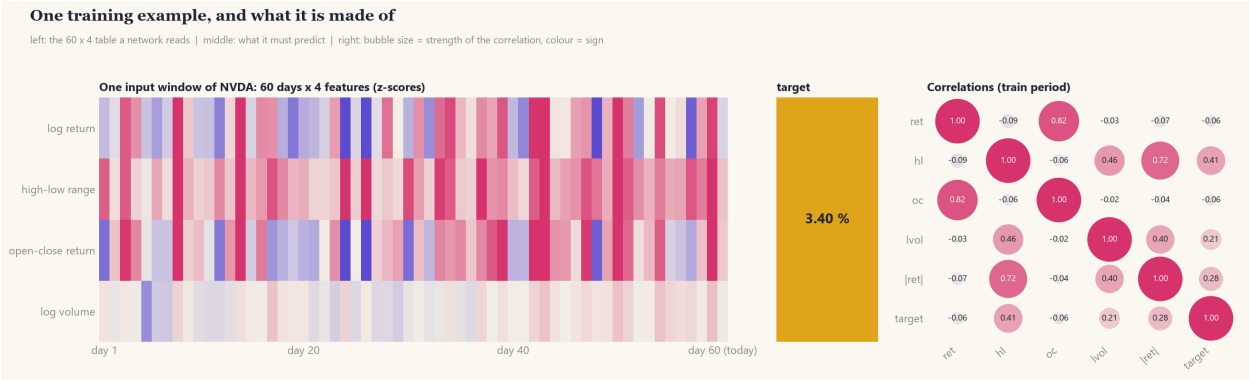


Figure 17. ST: one training window as a barcode with its target (left) and the correlations between the features and the target on the training period (right).

6 Implementation in PyTorch and Keras

6.1 The same network written twice

We use the two libraries in their usual way [10, 11]. Keras describes the network as a list of layers. Every recurrent layer except the last has to return its whole sequence of states (`return_sequences=True`), because the next layer needs a sequence as input; the last layer returns only its final state $h_T^{(5)}$. PyTorch has one module with `num_layers=5` that returns the top layer's states for all steps, and the forward pass picks the last one. The code for EC is below; ST differs only in the input (four features instead of an embedding), the size of the hidden state, and the output (no sigmoid, squared error instead of cross-entropy).

Keras

```
cell = {"RNN": layers.SimpleRNN, "LSTM": layers.LSTM}[kind]
stack = [layers.Input((MAXLEN,), dtype="int32"), layers.Embedding(VOCAB, EMB)]
stack += [cell(H, return_sequences=(l < LAYERS - 1)) for l in range(LAYERS)] # layers 1-4:
    all steps; 5: last
stack += [layers.Dense(1, activation="sigmoid")] # g = sigmoid
model = keras.Sequential(stack)
model.compile(optimizer=keras.optimizers.Adam(LR, clipnorm=CLIP), loss="binary_crossentropy")
```

PyTorch

```
class TorchSentiment(nn.Module):
   def __init__(self, kind):
      super().__init__()
      self.emb = nn.Embedding(VOCAB, EMB)
      self.rnn = {"RNN": nn.RNN, "LSTM": nn.LSTM}[kind](EMB, H, num_layers=LAYERS,
          batch_first=True)
      self.head = nn.Linear(H, 1)
   def forward(self, x): # x: (B, T) word ids
      out, _ = self.rnn(self.emb(x)) # out: (B, T, H), top-layer states
      return self.head(out[:, -1]).squeeze(1) # logit from h^(5)_T; sigmoid inside
          BCEWithLogitsLoss
```

The embedding deserves a word of explanation. A word id is only a label: id 171 is not "bigger" or "more negative" than id 170. The embedding layer is a lookup table with 20,000 rows of 64 numbers that replaces each id by its row. The rows start random and are trained together with

the rest of the network, so words used in similar ways end up with similar vectors; this hands the recurrent layers meaningful numbers instead of arbitrary labels, and it is where most of the model's parameters live (1.28 of 1.32 million in the EC RNN).

6.2 Weight names and the numerical check

The two libraries store the same numbers under different names and in different shapes, which is the main trap when comparing them (Table 10).

Table 10. The same weights under three names.

This report	Keras	PyTorch
W_x ($F \times H$, or $F \times 4H$)	`kernel`	`weight_ih_l{k}`, stored transposed
W_h ($H \times H$, or $H \times 4H$)	`recurrent_kernel`	`weight_hh_l{k}`, transposed
b	`bias`	`bias_ih_l{k}` + `bias_hh_l{k}`
gate order	i, f, c, o	i, f, g, o

To check that both frameworks compute the compositions of §3 and §4, the concepts notebook takes the weights of a PyTorch 5-layer network ($F = 3$, $H = 4$, $T = 6$, a batch of 2 sequences), copies them into a NumPy implementation written directly from the equations (using only matrix products, additions, tanh, sigmoid and $\odot$) and into a Keras stack of five layers, and compares the states of the top layer step by step. If all three agree, the libraries do exactly what we wrote down.

Table 11. Largest absolute difference of $h_t^{(5)}$ over all steps and sequences (float32 arithmetic).

	hand-written vs PyTorch	hand-written vs Keras
5-layer RNN	$6.7 \cdot 10^{-8}$	$9.2 \cdot 10^{-8}$
5-layer LSTM	$4.6 \cdot 10^{-8}$	$8.0 \cdot 10^{-8}$

The differences are about 10^{-7}, which is the size of single-precision rounding (float32 carries about seven decimal digits), so the frameworks compute exactly the functions written in §3–§4. The parameter counts of Table 5 confirm it from another side: our formula and the Keras count agree to the last digit, and PyTorch differs only by its second bias vector.

6.3 The same starting point in both frameworks

Weights are not zero at the start of training; they are drawn from a distribution, and the distribution matters. Keras draws the input weights from a Glorot-uniform distribution [7] (a range that depends on the matrix size, chosen so that the signal keeps a similar size from layer to layer), the recurrent weights as an orthogonal matrix [8] (a matrix that neither shrinks nor stretches a vector, which is the best possible start for a repeated multiplication), the biases as zero and, for the LSTM, the forget-gate bias as 1; the embedding is drawn uniformly in ± 0.05. PyTorch's defaults are different (uniform in $\pm 1/\sqrt{H}$ and forget bias 0), so the PyTorch models are re-initialised block by block, each of the four gates separately. The reason is the one the gradient experiment gave: initialisation decides how much gradient flows at the start (Figure 8), and with different defaults a difference between frameworks could be mistaken for a difference between architectures.

7 Experimental Protocol

7.1 One recipe for all eight runs

A comparison is only fair if everything except the thing being compared is identical. If the LSTM beat the RNN but the two had been trained with different optimizers, we could not tell whether the architecture or the optimizer made the difference. All eight runs therefore share one random seed (42) for Python, NumPy, TensorFlow and PyTorch, the same data arrays, the same architecture shape and the same training recipe (Table 12), explained below.

Table 12. One protocol for all runs.

Setting	EC	ST
layers / hidden units H	5 / 64	5 / 32
sequence length T	100 words	60 days
input	embedding $20{,}000 \times 64$	4 z-scored features
output	1 unit, sigmoid	1 unit, linear
loss	binary cross-entropy	mean squared error (standardised target)
optimizer	Adam, learning rate 10^{-3}, gradient clip-norm 1, batch size 256	
stopping	at most 10 epochs; early stopping on validation loss, patience 2, best weights restored	
seed	42 for Python, NumPy, TensorFlow and PyTorch	

- **Adam, learning rate** 10^{-3} [6]. Training nudges every weight a little in the direction that lowers the loss, and Adam is a standard way of choosing the size of each nudge: it keeps a running average of every weight's recent gradient and of its square, so weights with steady gradients move confidently and weights with noisy gradients move cautiously. The learning rate scales all the steps.
- **Gradient clipping at norm 1** [3]. In a recurrent network a single bad batch can produce an enormous gradient that throws the weights far off; clipping rescales the gradient so that its length never exceeds 1, and only acts in these rare emergencies.
- **Batch size 256.** The network looks at 256 examples, computes the average loss on them and makes one update. Larger batches give smoother updates, smaller ones more updates per pass over the data; 256 is a common compromise that also keeps a CPU busy.
- **Binary cross-entropy** for EC, $-[y \ln p + (1-y)\ln(1-p)]$: for a positive review ($y = 1$) a model that says $p = 0.9$ pays 0.105, one that says $p = 0.1$ pays 2.30, so confident mistakes are punished hard. **Mean squared error** for ST, on the target standardised to mean 0 and standard deviation 1; a model that always predicts the training mean then has a loss of about 1.
- **At most 10 epochs with early stopping (patience 2).** An epoch is one pass over the training data. After each epoch the loss on the validation data, which the network never trains on, is measured. A network with more than a million parameters will sooner or later start to memorise its training examples, and then the validation loss rises while the training loss keeps falling. Early stopping watches for this: if the validation loss has not improved for two epochs, training stops and the weights of the best epoch are kept.

7.2 Baselines: what a network has to beat

A score means little without something to compare it with.

EC. The *majority class* (always answer "positive") uses no information at all, so its accuracy of 0.70 is the floor, and its F1 for the negative class is exactly 0. The second baseline is *TF-IDF with logistic regression*, a strong classical text classifier and the fair opponent of a recurrent network because it ignores word order completely. Each review becomes a long vector with one entry per word or per word pair (a *bigram* such as "not good"), and each entry is the **TF-IDF** weight of that term: how often it occurs in this review (damped with a logarithm, so that saying a word ten times does not count ten times) multiplied by how rare it is across all reviews. A word that occurs in nearly every review ("the") gets a weight close to zero, a word frequent in this review but rare overall ("defective") gets a large one. A logistic regression then learns one weight per entry and adds up the weights of the entries present: a weighted vote of the words. It trains in seconds, and if a recurrent network cannot beat it, the ability to read word order is not paying off.

ST. The *train mean* (always predict the average $|r|$ of the training period) is the "know nothing" forecast and the zero point of R^2_{oos}; *persistence*, $\hat{y} = |r_t|$, uses only the last day; the *20-day mean* of $|r|$ is the classical volatility estimator, which uses a bit of memory with no learning at all.

7.3 Metrics

EC. All metrics start from the predictions on the test reviews, where a probability of at least 0.5 means "positive". *Accuracy* is the share classified correctly: easy to understand, but misleading when 70 % of the reviews are positive. *Recall* of the negative class asks "of all truly negative reviews, how many did we find?" and *precision* "of the reviews we called negative, how many really are?" (of 100 reviews, 30 negative; a model finds 24 and wrongly flags 6 positive ones: recall $24/30 = 0.8$, precision $24/30 = 0.8$). A shop that wants to find unhappy customers needs both. The *F1 score* $F_1 = 2PR/(P+R)$ is the harmonic mean of precision and recall, high only if both are high; the **F1 of the negative class** tells us how well the minority class is handled, and the **macro F1**, the mean of the F1 of both classes, lets the small negative class count as much as the large positive one. **ROC-AUC** is the probability that a randomly chosen positive review gets a higher score than a randomly chosen negative one (0.5 is guessing, 1 is perfect ranking); it does not depend on the 0.5 threshold, so it judges the quality of the scores themselves.

ST. Errors are reported in the original unit, the absolute daily log return, in percent. The **RMSE** $= \sqrt{\frac{1}{N}\sum(y-\hat{y})^2}$ is the typical size of a miss with big misses weighted more, and the **MAE** $= \frac{1}{N}\sum|y-\hat{y}|$ is the plain average miss. The **out-of-sample** R^2 $R^2_{\text{oos}} = 1 - \sum(y-\hat{y})^2 / \sum(y-\bar{y}_{\text{train}})^2$ is the share of the squared error that the model removes compared with always predicting the training average: 0 means no better than that constant, 1 would be perfect, and a negative value means worse. For daily volatility, values of a few percent to ten percent are normal, because day-to-day noise is huge. The **correlation** between forecast and truth ignores any offset or scale and asks only whether the forecast rises and falls with the truth.

7.4 Machine

The dataset notebooks were run on a Windows PC on the CPU (TensorFlow 2.22.0-rc0, PyTorch 2.14.1 for the CPU), the four runs of a dataset one after the other. Times compare the models on this machine only.

8 Results: Review Sentiment (EC)

8.1 The scores

All four networks and both baselines were evaluated on the same 22,219 test reviews, which no model saw during training or early stopping (Table 13, Figure 18). Looking at the rows, all four recurrent networks land between 0.918 and 0.931 accuracy, far above the majority baseline (0.70) but, to our surprise, *below* the TF-IDF baseline (0.941). Among the networks the pattern is the same in both frameworks: **the LSTM beats the RNN**, by 0.8 points of accuracy in Keras (0.9301 against 0.9225) and 1.3 points in PyTorch (0.9311 against 0.9183), and the gap grows for the negative class, where the LSTM's F1 is higher by 0.9 points in Keras and 2.7 points in PyTorch. The ROC-AUC tells the same story (0.9745 against 0.9707, and 0.9760 against 0.9676). The two frameworks agree on accuracy within half a point for the same architecture, so the difference between RNN and LSTM is larger than the difference between libraries: it is a property of the models, not of the software.

Table 13. EC test set (22,219 reviews). Bold: best network. Epochs: number trained (best epoch kept).

Run	Accuracy	F1 macro	F1 neg.	ROC-AUC	Params	s/epoch	Epochs (best)
majority class	0.7006	0.4120	0.0000	0.5000	–	–	–
TF-IDF + log. regression	0.9414	0.9298	0.9011	0.9825	–	–	–
`EC-TF-RNN`	0.9225	0.9084	0.8726	0.9707	1,321,345	43	4 (2)
`EC-TF-LSTM`	0.9301	0.9161	0.8817	0.9745	1,445,185	198	4 (2)
`EC-PT-RNN`	0.9183	0.8999	0.8570	0.9676	1,321,665	42	4 (2)
`EC-PT-LSTM`	**0.9311**	**0.9176**	**0.8844**	**0.9760**	1,446,465	93	5 (3)

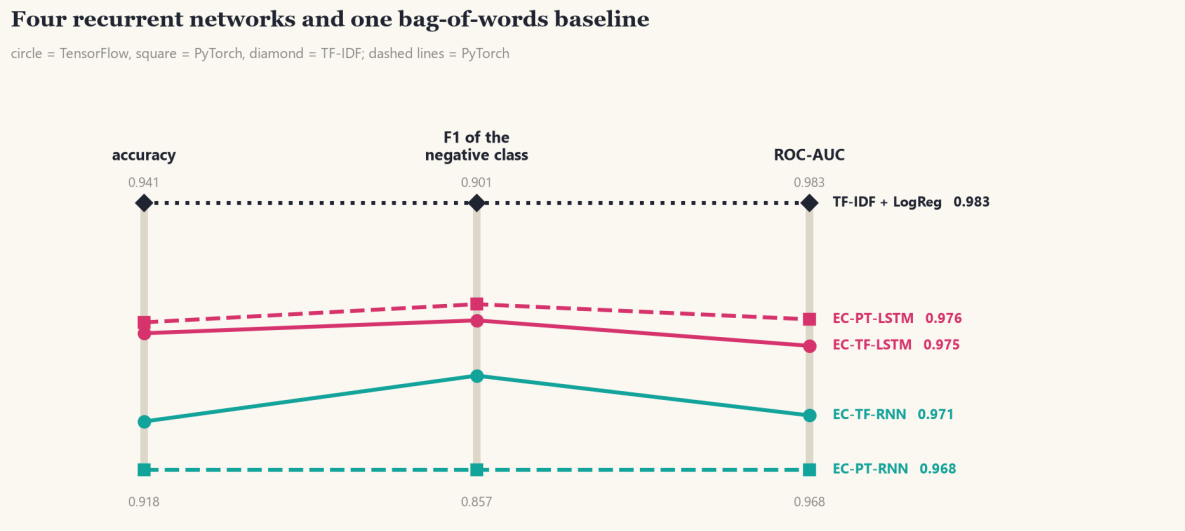


Figure 18. EC: the four networks and the bag-of-words baseline on three metrics (circle = Keras, square = PyTorch, dashed = PyTorch). Each LSTM (magenta) is above the RNN of the same framework (teal) on every metric, and the TF-IDF baseline is above all four.

8.2 Learning curves: overfitting after two or three epochs

Both Keras networks reach their lowest validation loss after 2 epochs, with 0.214 (RNN) and 0.194 (LSTM). After that the validation loss rises (to 0.239 and 0.243 at epoch 4) while the training loss keeps falling (the RNN's from 0.30 to 0.15). This is overfitting in action: from epoch 3 on the networks fit the particular training reviews better and unseen reviews worse. The cause is visible in Table 5: of the 1.32 million parameters of the RNN, 1.28 million are the embedding table, which learns 64 numbers for each of 20,000 words from 104,000 reviews and can memorise a great deal; rare words are memorised quickly. Early stopping notices after two epochs without improvement and restores the weights of the best epoch, so the test scores come from epoch 2. In PyTorch the RNN's best epoch is 2 (validation loss 0.220) and the LSTM keeps improving until epoch 3 (0.203) before stopping at epoch 5; its validation loss is bumpy (0.208, 0.226, 0.203, 0.203, 0.210), which shows that a single validation number is a noisy guide: which epoch is "best" depends partly on chance. In every pair the LSTM's best validation loss is lower than the RNN's of the same framework (0.194 against 0.214 in Keras, 0.203 against 0.220 in PyTorch).

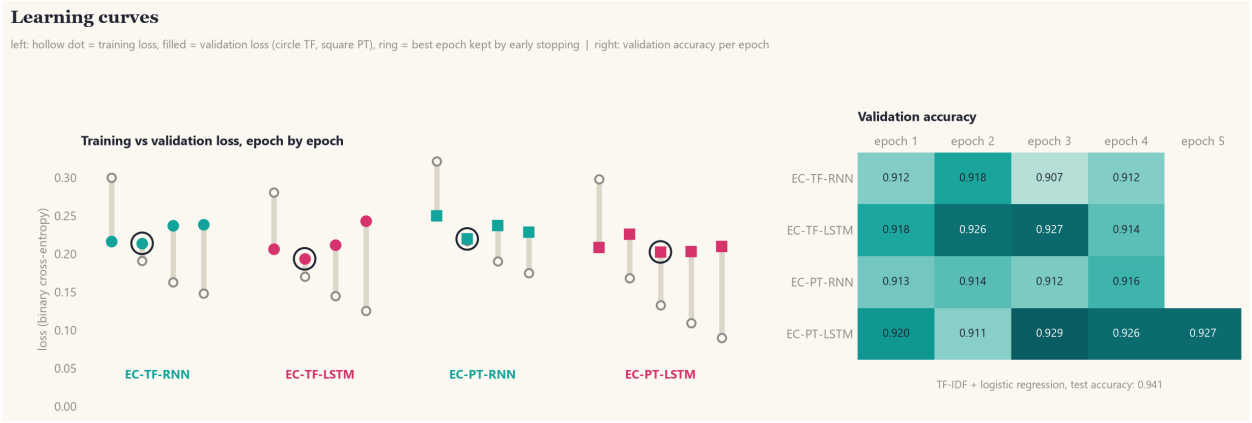


Figure 19. EC: training loss (hollow dot) and validation loss (filled dot) per epoch for each run, with the epoch kept by early stopping ringed (left), and the validation accuracy per epoch (right).

8.3 Where do the errors go?

Accuracy hides which mistakes are made, so Figure 20 shows, for each run, the share of the truly negative reviews that is called positive and of the truly positive reviews that is called negative, and the ROC curve of each run. All four runs mislabel a larger share of the *negative* reviews than of the positive ones, which is the price of being the minority class: the Keras RNN lets 757 of the 6,653 negative test reviews pass as positive (11.4 %), the Keras LSTM 869 (13.1 %), the PyTorch LSTM 798 (12.0 %) and the PyTorch RNN as many as 1,213 (18.2 %), while only 3.9 to 6.2 % of the 15,566 positive reviews are called negative. A shop that wants to catch unhappy customers therefore misses between one in nine and one in five of them with any of these models.

The architectures trade the two kinds of mistake differently. In Keras the LSTM catches fewer complaints than the RNN (13.1 % missed against 11.4 %) but raises far fewer false alarms (4.4 % of the positive reviews against 6.2 %), so its precision on the negative class is higher at a similar recall. In PyTorch it is the other way round for the complaints: the LSTM misses 12.0 % against 18.2 % for the RNN, at the price of slightly more false alarms (4.7 % against 3.9 %). The PyTorch RNN is the weakest run on negative reviews, which is what its low negative-class F1 (0.857) said. Which profile is better depends on the shop, so the F1 of the negative class, which weighs both, is the fair summary. In the ROC panels the order of the AUC values is visible as the order of the curves:

the LSTM curves lie closer to the top-left corner than the RNN curves, and the dotted TF-IDF curve (AUC 0.9825) lies highest overall.

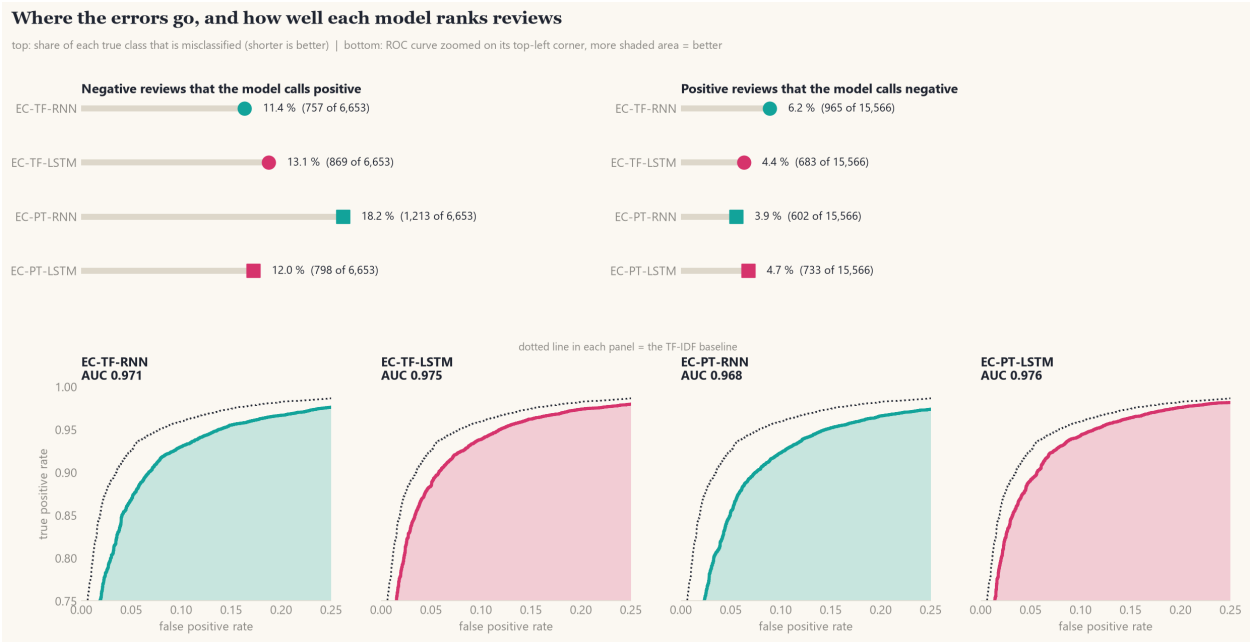


Figure 20. EC: share of each true class that is misclassified (top, shorter is better) and ROC curves zoomed on the top-left corner (bottom; dotted = TF-IDF baseline, shaded area = AUC).

8.4 How confident are the models?

A model can be right for the wrong reason, and two models with the same accuracy can differ in how sure they are. For every test review we take the **log-odds the model gave to the true class**, $\ln \frac{p}{1-p}$ with p the probability assigned to the correct label: above 0 the model was right, below 0 wrong, and the further from 0 the more confident it was. The scale is easy to translate: 0 is a coin toss, 2 means a probability of 0.88 for the true class, 3 means 0.95, 4.6 means 0.99. In Figure 21 the right halves of the violins (positive reviews) lie higher than the left halves (negative reviews) in every run: the median positive review receives log-odds between 4.6 and 5.9 (probability 0.99–0.997), the median negative review only between 2.0 and 3.6 (0.88–0.97). The models are much surer of themselves on praise than on complaints, which fits the fact that complaints are the minority class and are more varied and longer.

The runs differ in how confident they are, and this does not simply follow their accuracy. The PyTorch RNN is the least sure on negative reviews (median 2.0; only 13 % of them receive more than 98 % for the right class), in line with its high miss rate. The PyTorch LSTM is the most confident of all (median 3.6 and 5.9; 41 % of the negative and 82 % of the positive reviews receive more than 98 %), but confidence has a price: it also has the largest share of negative reviews that it gets wrong *with confidence*, 2.3 % of them with less than 2 % for the true class, against 0.9–1.4 % for the Keras runs and 1.1 % for the PyTorch RNN. In the Keras pair the LSTM is somewhat *less* sure than the RNN (30.7 % against 37.8 % of the negative reviews above 98 %) and still more accurate, with fewer confident errors (0.9 % against 1.4 %). A higher accuracy therefore does not come from the model being more certain, and a model can be sure and wrong, so a deployed system should not read its probability as a guarantee.

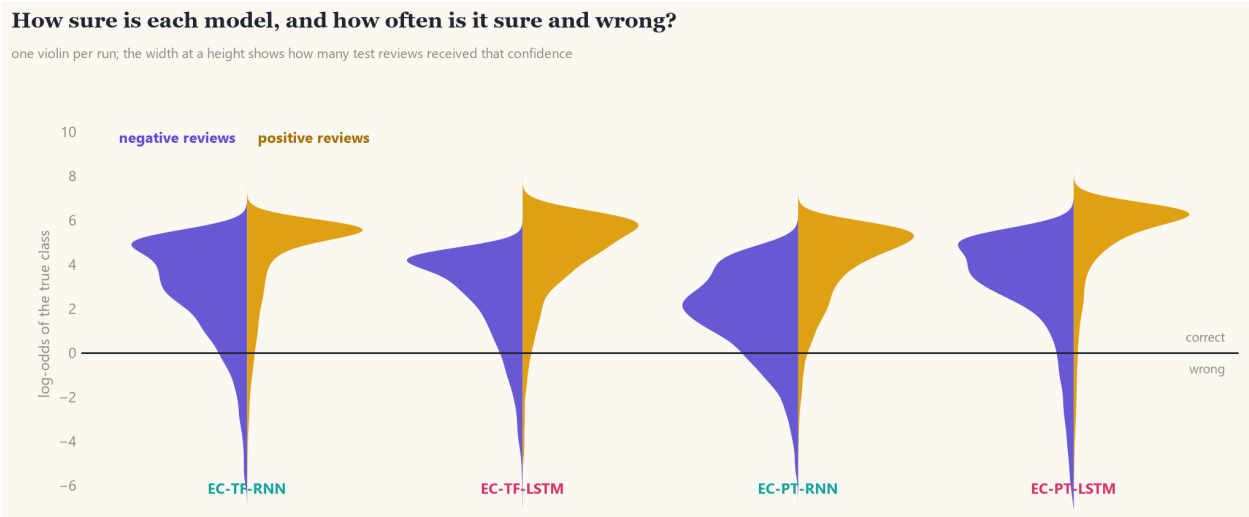


Figure 21. EC: the log-odds that each run gave to the true class of every test review (left half = negative reviews, right half = positive reviews). Mass below the black line are mistakes; the long tails below -4 are mistakes made with confidence.

8.5 Does the LSTM read long reviews better?

This is the central question of the assignment. If the RNN loses information over many steps, its accuracy should fall faster on long reviews than the LSTM's. Table 14 and Figure 22 give the test accuracy in six length buckets (the number of words before cutting at 100). Remember the pre-padding: in a 10-word review the first 90 steps are padding and the 10 real words come last, so a short review needs almost no memory at all.

Table 14. EC: test accuracy by review length (words before cutting to 100). The Δ column is LSTM $-$ RNN averaged over the two frameworks.

Words	EC-TF-RNN	EC-TF-LSTM	EC-PT-RNN	EC-PT-LSTM	LSTM − RNN	Reviews
1–10	0.950	0.952	0.945	0.954	+0.006	5,955
11–20	0.929	0.936	0.920	0.938	+0.013	3,697
21–40	0.926	0.937	0.926	0.936	+0.011	5,546
41–70	0.914	0.925	0.907	0.923	+0.014	3,070
71–100	0.911	0.919	0.907	0.918	+0.010	1,458
>100 (cut)	0.855	0.868	0.855	0.874	+0.016	2,493

Accuracy falls with length for every model, from about 95 % on reviews of 1–10 words to 85–87 % on reviews over 100 words. Long reviews are harder for everyone, partly because they are cut at 100 words and partly because they contain mixed opinions. The lower half of the figure shows the LSTM ahead of the RNN in every bucket, and the advantage is smallest where we expect no memory problem at all: 0.1 points (Keras) in the shortest bucket against 1.3 points in the longest, and 0.8 against 1.8 in PyTorch. So the direction is the one the theory predicts, a gain that grows with length, but the trend is modest and not perfectly smooth (in Keras the gain is 1.1 points at 41–70 words and 0.8 at 71–100). The right reading is that the LSTM helps a little everywhere and somewhat more on long reviews, not that the RNN fails on them. A likely reason for the modest size of the effect is that even a long review decides its verdict with a few words, and an RNN whose gradient fades after twenty or thirty steps can still learn to attend to the end of the review; the long-range advantage of the LSTM is real but needs much longer dependencies than one hundred

words to become dramatic.

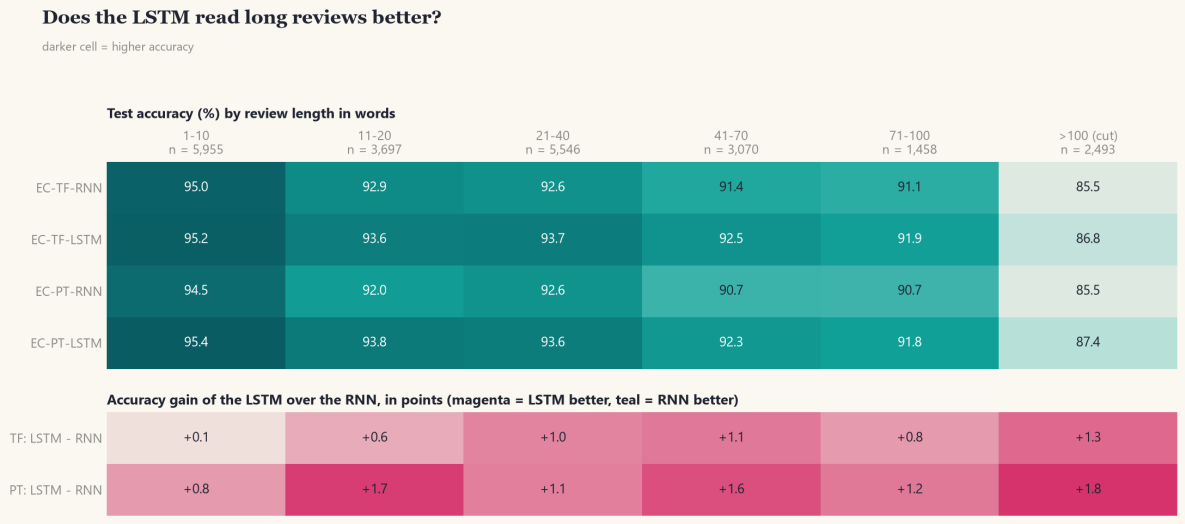


Figure 22. EC: test accuracy by review length (top) and the gain of the LSTM over the RNN in points (bottom; magenta = LSTM better, teal = RNN better).

8.6 Why does a bag of words win?

The strongest model of the experiment is the baseline: TF-IDF with logistic regression reaches 0.941 accuracy, 0.930 macro F1 and 0.983 AUC, better than all four networks on every headline metric, after 9 seconds of training instead of several minutes. We report this negative result as measured and explain it in three parts. (a) The signal in these reviews is mostly lexical: words like *refund*, *defective* and *excellent* decide the label (Figure 12), and a bag of words sees all of them in every review, with a convex training problem that has a single best answer. (b) The networks see only the first 100 words and must learn a 64-dimensional embedding of 20,000 words from 104,000 reviews, which is why they overfit after two or three epochs, whereas the linear model has no such problem. (c) We gave the networks no regularisation and no pretrained word vectors, to keep the experiment about the architectures. Word order helps (the LSTM beats the RNN), but not enough to make up for this.

8.7 Reading examples

Numbers hide the actual reviews, so Table 15 shows some. The Keras RNN and LSTM give different answers on 782 of the 22,219 test reviews (3.5 %). On all test reviews the LSTM is right on 20,667 and the RNN on 20,497, a difference of 170, so among the 782 disagreements the LSTM must be right on 476 (61 %) and the RNN on 306. The reviews in the table were drawn at random from the disagreements.

Table 15. EC: disagreements of `EC-TF-RNN` and `EC-TF-LSTM` on the test set (P = probability of "positive").

Review (start)	Stars	P RNN	P LSTM
"Love the phone, but it drains the battery fairly quickly. Very slick, therefore easy to lose grip…"	4	0.494	0.834
"I think it was a good phone but my teenage son…broke it after having it only two weeks."	4	0.287	0.534
"…there is not problem with this jet."	5	0.360	0.547
"I ordered a brand new samsung galaxy s4…the phone delivered to me is 'used' one…I am very disappointed…"	1	0.669	0.378
"I'm changing my review from 4 stars to 2. This Deluxe Edition 128gb does not work with U.S. carriers…"	2	0.437	0.649
"I think peoples' expectations at this price point are out of sync. This is a great tablet with LTE and calling abilities…"	2	0.473	0.925

Almost all of them are *mixed*. "Love the phone, but it drains the battery", "I think it was a good phone but my teenage son…broke it" and "there is not problem with this jet" start or end with praise, or use a negation, and the RNN, which relies on the last words it read, is swayed by the complaint-like words (*drains*, *broke*, *not problem*) and answers negative, whereas the LSTM keeps the earlier verdict and answers positive. The negation case is the one in which we expected a model with a memory to help, and it does, although the 0.53 and 0.55 show how narrow the margin is. The Samsung buyer who received a "used" phone starts with neutral product details and ends with *very disappointed*; here the LSTM is right and the RNN wrong. The last two reviews are LSTM mistakes and are instructive. The tablet review carries 2 stars but reads as praise, so the LSTM's 0.93 is a defensible reading of the text and the label is the questionable part. The reviewer who is "changing my review from 4 stars to 2" opens with a rating and a product description before the complaint, and the early, neutral content outweighs the later complaint in the LSTM, while the RNN, reading the end, is right. These are the reviews at which any model is in trouble: the star rating and the text disagree, or the verdict is spread over a long text.

9 Results: Stock Volatility (ST)

9.1 Baselines first

On the test period (2022-09 to 2024-12, 81,925 windows) persistence is *worse* than the train mean (Table 16): its R^2_{oos} is -0.70 and its RMSE 1.78 % against 1.36 %. One day's move is a noisy estimate of tomorrow's, because a single large or small move is partly luck: a 5 % day is often followed by a 1 % day. Averaging 20 days removes much of that noise and beats the mean (RMSE 1.33 %, R^2_{oos} 0.044, correlation 0.25). That is the volatility clustering of §5 put to work with no learning at all, and the recurrent networks have to beat the 20-day mean to show that they have learned something more than a moving average.

Table 16. ST test period (81,925 windows). Errors in percent of the daily move. Bold: best network.

Run	RMSE (%)	MAE (%)	R^2_{oos}	Corr.	Params	s/epoch	Epochs (best)
train mean	1.3632	0.8809	0.0000	–	–	–	–
persistence $\lvert r_t \rvert$	1.7783	1.1281	-0.7018	0.149	–	–	–
20-day mean $\lvert r \rvert$	1.3326	0.8656	0.0443	0.251	–	–	–
`ST-TF-RNN`	1.3015	0.8419	0.0884	0.300	9,537	51	3 (1)
`ST-TF-LSTM`	1.2990	**0.8158**	0.0920	0.311	38,049	138	6 (4)
`ST-PT-RNN`	1.3025	0.8238	0.0871	0.299	9,697	46	4 (2)
`ST-PT-LSTM`	**1.2957**	0.8399	**0.0965**	0.311	38,689	70	5 (3)

9.2 The networks beat every baseline, by a small margin

All four networks beat all three baselines on every metric: the RMSE falls from 1.3326 % (20-day mean) to between 1.2957 % and 1.3025 %, and the R^2_{oos} roughly doubles, from 0.044 to between 0.087 and 0.097, with a correlation of 0.30–0.31 between forecast and truth (Figure 23). So the networks learned something real that goes beyond a moving average: they weigh the recent days adaptively instead of equally, and they use the range and volume features, which carry information about volatility that $\lvert r \rvert$ alone does not. In absolute terms, however, the gain is small. The networks explain about 9 % of the squared error of tomorrow's move, which is the typical size for this problem and a reminder that most of the move of a single day is news that no price history contains.

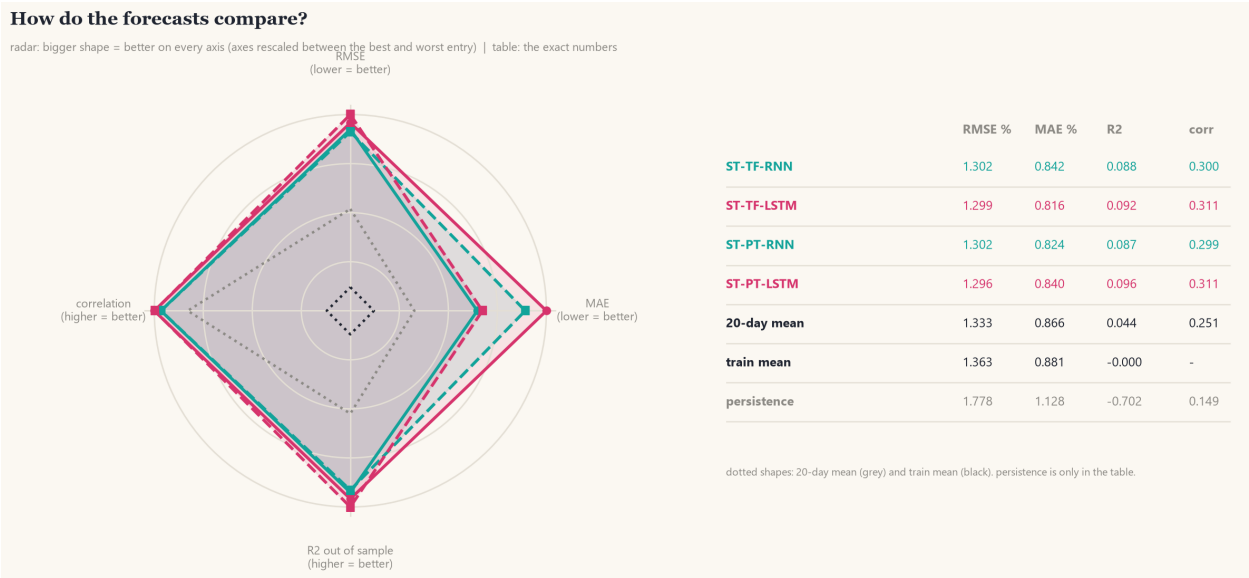


Figure 23. ST: the four networks and the baselines on RMSE, MAE, R^2 and correlation, each axis stretched so that the best entry reaches the rim (RMSE and MAE inverted; radar, left) and the exact values (right). The four network shapes lie almost on top of each other, outside the two baselines.

9.3 The LSTM is only marginally better than the RNN

The LSTM is slightly ahead of the RNN on RMSE, R^2 and correlation in both libraries (R^2 0.092 against 0.088 in Keras, 0.097 against 0.087 in PyTorch; correlation 0.311 against 0.300 and 0.311 against 0.299), but the differences, between 0.002 and 0.007 percentage points of RMSE, are within the noise of a single run, and on MAE the order is not even consistent (the Keras LSTM is better, the PyTorch LSTM slightly worse). The picture says in one glance what the numbers say: *recurrent*

versus not recurrent matters, *RNN versus LSTM* barely does. The explanation fits the data. The autocorrelation of $|r|$ falls from 0.215 to below 0.05 inside the 60-day window, so the useful information sits in the last few weeks, which the RNN's gradient can still reach, and the LSTM's long memory has little extra to find. Meanwhile the LSTM has four times the parameters (38,049 against 9,537) and needs 1.5 to 2.7 times as long per epoch.

9.4 Learning curves and the regime shift

The learning curves are flat (Figure 24). In Keras the RNN's best validation loss is already reached in epoch 1 (0.875) and gets worse afterwards, the LSTM improves slowly until epoch 4 (0.873) and then stays level, while the training loss keeps falling (the RNN's from 0.785 to 0.746, the LSTM's from 0.783 to 0.717). In PyTorch the RNN is best in epoch 2 (0.871) and the LSTM in epoch 3 (0.864). Two observations stand out. First, every validation loss (0.86 to 0.90) is far below the dotted line of 1.008, the loss of always predicting the training mean, so the networks beat the constant forecast on the validation period too. Second, the validation loss is higher than the training loss from the first epoch on, and the gap widens only slowly (from about 0.09 to 0.13–0.16): this is not only classic overfitting, it is also the *regime shift* of §5. The validation period (mid-2020 to autumn 2022: the recovery after Covid, inflation, rate rises) was more turbulent than the training period, so the same model makes larger errors there. Which epoch is "best" is therefore decided by wobbles of about 0.01 (the best epochs are 1, 4, 2 and 3), and a different seed could well choose another epoch. We keep this in mind and avoid interpreting differences between models that are smaller than such wobbles.

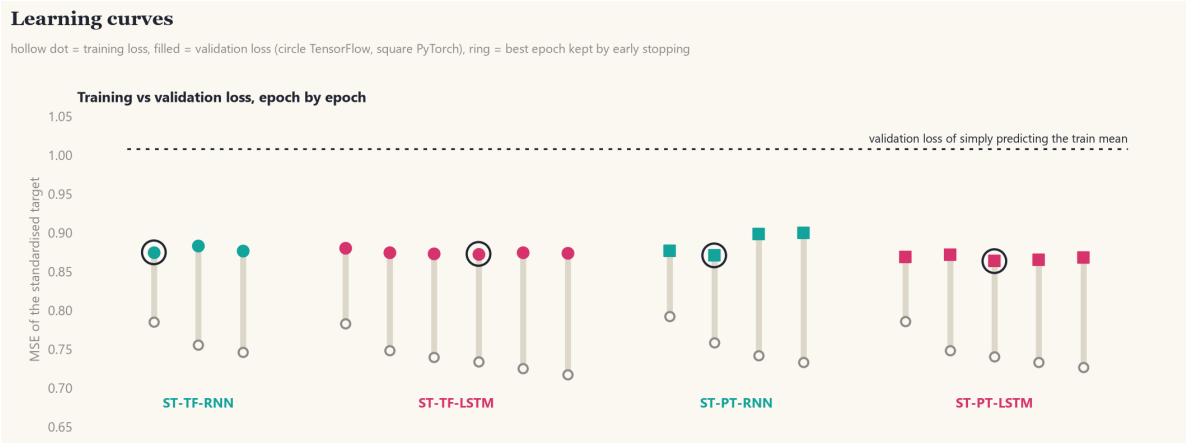


Figure 24. ST: training and validation loss per epoch for each run (the ring marks the epoch kept) with the loss of the constant forecast (dotted).

9.5 What the forecasts look like

Figure 25 shows, on top, the first stock (NVDA) over the last 250 test days: the true size of the next day's move, the 20-day mean and the Keras forecasts, all smoothed with a 10-day average so that the lines are readable (individual days are far too noisy to draw). With an R^2 of about 0.09 a forecast cannot follow the truth closely, and the picture reflects that: the forecast lines are much flatter than the true moves, because networks trained to minimise squared error on a very noisy target *hedge* towards the middle (the best forecast under squared error is the conditional mean of $|r_{t+1}|$, not a spike that nobody can foresee). The two hexagon charts below use all 81,925 test windows and plot the forecast against what actually happened, darker hexagons meaning more windows. Whatever the forecast, the realised moves spread widely (a forecast of 1.5 % is followed

by anything from almost nothing to several percent), but the coloured line, the average outcome for each tenth of the forecasts, rises along the diagonal: days with a higher forecast really are, on average, more turbulent. That monotone relationship is the useful content of the forecast. It ranks days by expected turbulence; it does not predict the size of any single day.

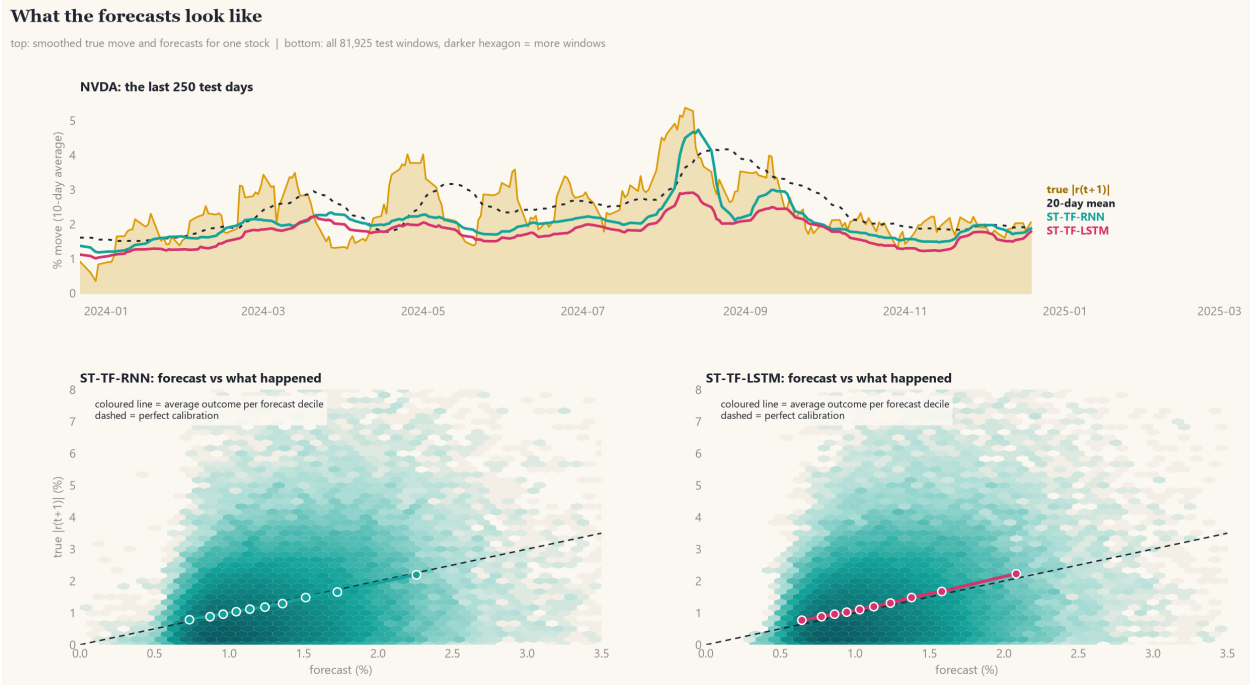


Figure 25. ST: true move, 20-day mean and Keras forecasts for NVDA over the last 250 test days (top, 10-day averages), and forecast against outcome for all test windows (bottom; the coloured line is the average outcome per decile of the forecast, the dashed line is perfect calibration).

9.6 Where do the networks help? Calm against turbulent markets

If memory is what matters, the groups of days with different volatility are where it should show: in a stable market everyone forecasts "about the same as lately", while after a regime change a model with a longer memory or better features could do better. We sorted the test windows into five equally large groups by the 20-day volatility at the time of the forecast (Table 17, Figure 26). Errors grow steeply with the volatility level, as they must (the RMSE is about 1.0 % in the calmest group and 1.8 % in the most turbulent), simply because the moves to be forecast are bigger. The gain of the networks over the 20-day mean grows in the same direction: from about 3 % of its error in the calmest group (and 1–2 % in the three middle groups) to 3.3 % for the RNNs and 3.5–4.0 % for the LSTMs in the most turbulent group. The networks therefore help most when markets are stormy, and the LSTMs help a little more than the RNNs there (4.0 % for the PyTorch LSTM against 3.3 % for the Keras RNN), the one place where the longer memory shows. The differences between models are nevertheless a few tenths of a percent of RMSE, small compared with the run-to-run noise.

Table 17. ST: test RMSE (%) by volatility level at prediction time (quintiles of the 20-day mean of $|r|$). The last column is LSTM − RNN averaged over the two frameworks.

20-day volatility	ST-TF-RNN	ST-TF-LSTM	ST-PT-RNN	ST-PT-LSTM	20-day mean	LSTM − RNN
calmest	0.9813	0.9829	0.9839	0.9810	1.0108	-0.0006
calm	1.0622	1.0596	1.0633	1.0617	1.0791	-0.0021
middle	1.2380	1.2354	1.2389	1.2326	1.2525	-0.0044
turbulent	1.3154	1.3123	1.3168	1.3098	1.3395	-0.0050
most turbulent	1.7650	1.7605	1.7648	1.7522	1.8248	-0.0086

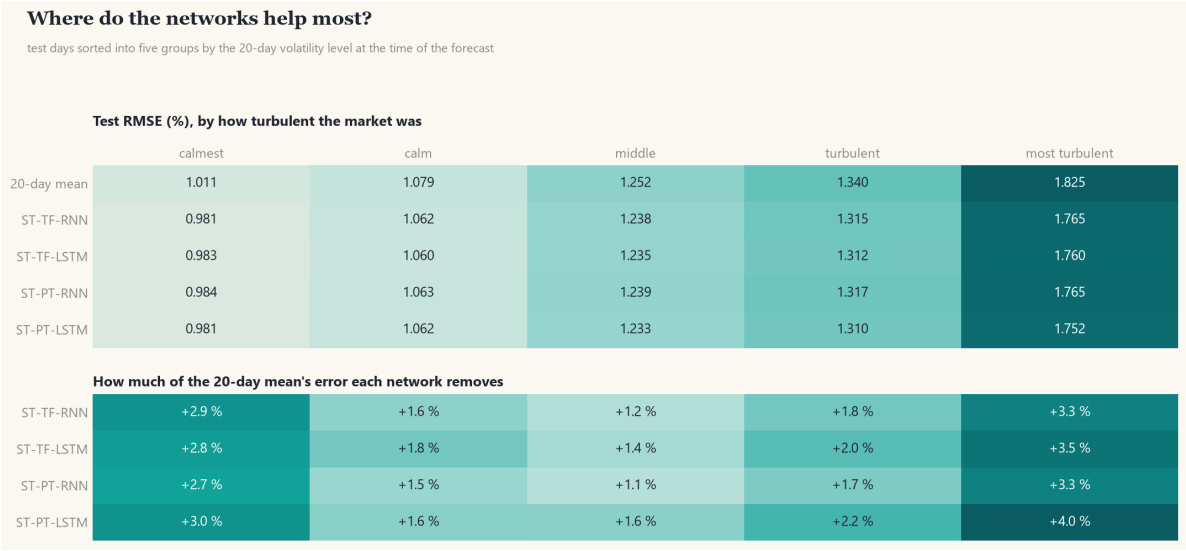


Figure 26. ST: test RMSE of each model in five volatility groups (top) and the percentage of the 20-day mean's error that each network removes (bottom).

10 Comparison: RNN against LSTM

The lecturer's criteria for comparing two architectures are accuracy, number of parameters, training time and behaviour. We take them in turn, with the eight runs of Table 1 as evidence.

Accuracy. The LSTM is better in all four pairs: the macro F1 on EC rises by $+0.0076$ (Keras) and $+0.0177$ (PyTorch), the R^2_{oos} on ST by $+0.0036$ and $+0.0094$. The gain is clear on EC and small on ST, and the size of the gain follows the length of the dependency that the task needs: long on EC (verdicts far from the end of the text, negations, a complaint after a polite opening), short on ST (a volatility memory of a few weeks, most of it in the last days). This is the prediction of §4 confirmed in two experiments, and the within-task evidence points the same way: on EC the LSTM's lead grows with review length (Figure 22), and on ST it grows with the turbulence of the market (Figure 26).

Parameters. The LSTM has four times the recurrent weights. On ST that is the whole model (9,537 $\rightarrow$ 38,049); on EC the shared embedding makes the total only 9 % larger (1.32 $\rightarrow$ 1.45 million). The extra parameters buy controlled memory, not simply capacity of the same kind: an RNN with four times as many units would still have the multiplicative gradient path, and it would not receive the $4.5 \cdot 10^{-4}$ of Figure 8.

Time. The LSTM takes 1.5 to 4.6 times as long per epoch on the CPU (Figure 27): 43 against 198 seconds on EC in Keras (4.6 times), 42 against 93 in PyTorch (2.2 times), 51 against 138 on ST in Keras (2.7 times) and 46 against 70 in PyTorch (1.5 times). Each step computes four gate products instead of one, plus the cell update, which explains why the LSTM is slower; the spread between the ratios shows that how much slower depends on how well each library optimises the LSTM on a CPU, because the RNN times of the two libraries are almost the same (43 and 42 seconds on EC) while the LSTM times are not. In the best case, the Keras LSTM on EC, an improvement of 0.8 points of macro F1 costs more than four times the training time per epoch.

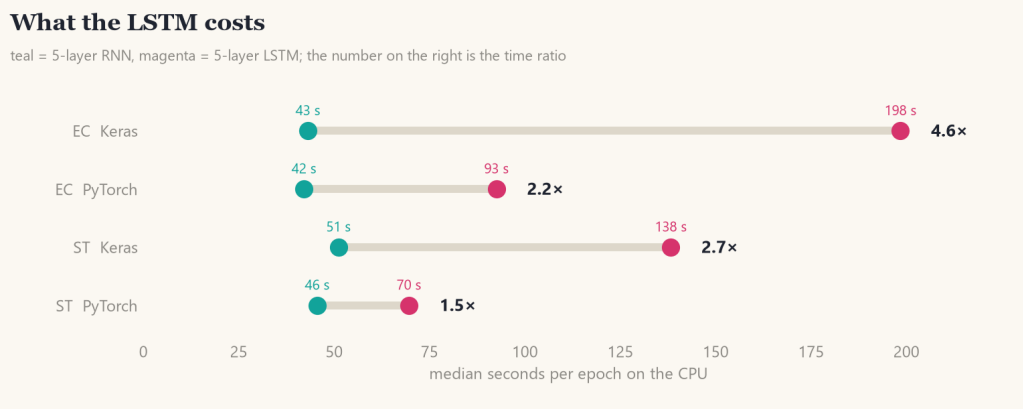


Figure 27. Median seconds per training epoch on the CPU for the eight runs (teal = RNN, magenta = LSTM) and the time ratio.

Behaviour. At initialisation the LSTM with forget bias 1 passes about five million times more gradient to the first step than the RNN (Figure 8), and after training its advantage concentrates where memory matters: long reviews and turbulent markets. The LSTM with forget bias 0 is no better than the RNN at the start, which is why the initialisation of the two frameworks had to be made identical.

Keras against PyTorch. The two frameworks compute the same function (§6) and give the same ranking everywhere, but their scores for the same architecture are not identical: they differ by up to 0.016 on the EC metrics (the negative-class F1 of the RNNs, 0.8726 against 0.8570) and by up to 0.005 in R^2 on ST. The data, the architecture, the starting distribution and the stopping rule are identical, but the actual random weights, the order of the mini-batches and the floating-point kernels are not, and early stopping then picks a different epoch (2 against 3 for the EC LSTMs, 1 against 2 for the ST RNNs). The difference between the libraries is therefore of the same order as the noise of a single run, and smaller than the difference between RNN and LSTM on EC, which is why we trust the EC conclusion and treat the ST conclusion as "small and consistent in sign, but not proven".

Table 18. Which architecture to reach for, as suggested by these experiments.

Situation	Suggestion	Evidence here
The answer depends on information many steps back (long texts, negation, delayed effects)	LSTM, with forget bias 1	EC: $+0.8$ to $+1.8$ points of macro F1, growing with length
The signal is mostly in the last few steps (short memory)	plain RNN is nearly as good at a quarter of the recurrent weights	ST: R^2 gain of 0.004–0.009
Words alone decide the label and training data is moderate	try a bag-of-words model first	EC: TF-IDF beats all four networks
Training time is tight	RNN, or a smaller LSTM	1.5–4.6 times per epoch

11 Discussion

11.1 What the experiments support

Three observations fit the theory of §3–§4 and none of them was imposed on the data. First, the *measured gradient* of the RNN falls by a factor of about 0.62 per step (ten orders of magnitude over 50 steps), and the cell path of an LSTM with open forget gates falls by about 0.94 per step. Second, the LSTM's advantage *follows the length of the dependency*: largest on long reviews and in turbulent markets, smallest on short reviews and in calm markets. Third, the *LSTM without the forget-bias trick is not better than the RNN at the start of training*, which is why the same recipe has to be used in both frameworks. These three together are the strongest evidence for the claim that the gated additive cell, and not just a larger model, is responsible for the improvement.

11.2 What surprised us

The most important surprise is the bag of words. We expected the LSTM to win on a language task, and it wins against the RNN, but not against a method that ignores the order of the words. The explanation of §8 (sentiment sits in single words, and the networks must learn their word vectors from scratch) points to the direction of the fix, not to a failure of the LSTM. The second surprise is how *small* the gain from memory is on ST; here the reason is in the data, because the memory of volatility fades within weeks, and not in the architecture. The third is the difference between libraries: the same architecture trained with the same recipe in two frameworks differs by up to 1.6 points of F1 on the negative class, which is a warning against reading a one-point difference between two runs as a finding.

11.3 Threats to validity

- **One run per configuration.** There are no repeated seeds, so differences below about 0.005 (for example the ST gap between RNN and LSTM in Keras) are within run-to-run noise. The EC conclusion rests on two frameworks agreeing in sign and on the length trend, not on any single difference.
- **EC reads at most 100 words, left to right, with embeddings learned from scratch.** Pre-trained embeddings or a bidirectional LSTM would probably close the gap to TF-IDF, and the 11 % of the reviews cut at 100 words lose the end, where the verdict is often found.
- **Labels are noisy.** A star rating is not always what the text says (the tablet review of Table 15),

so some of the remaining errors are errors of the label, not of the model.

- **No tuning per model.** The size H, the learning rate and the depth are the same for both architectures, so each might do better with its own settings.
- **ST data.** Survivorship bias in the choice of stocks, a regime shift between training, validation and test, four hand-chosen features and no market-wide inputs, and only the next day is forecast.
- **CPU timing on one machine.** On a GPU with cuDNN the time ratio would be different, and the ratio would depend on the library even on the CPU, as the two frameworks show.

12 Conclusion and Future Work

Written as compositions of matrix product, addition, tanh, sigmoid and element-wise product, the 5-layer RNN and the 5-layer LSTM differ in a single mechanism: the LSTM's additive, gated cell. The chain rule shows why that matters (the RNN's gradient is a product of shrinking factors, the LSTM's cell gradient a product of forget gates), the measured gradient maps confirm it ($8.7 \cdot 10^{-11}$ against $4.5 \cdot 10^{-4}$ at the first step), and they add a warning: with the forget gate closed at the start, the LSTM is no better than the RNN. PyTorch and Keras compute exactly these compositions, to about 10^{-7}, and their parameter counts agree with a hand formula.

On real data the mechanism pays off in proportion to how far back the needed information lies. On e-commerce reviews the LSTM is clearly better than the RNN, most so on long reviews, although a bag-of-words baseline is still better than both networks. On stock volatility both networks roughly double the R^2 of the classical 20-day estimate, and the LSTM adds little except in turbulent periods. The LSTM costs four times the recurrent parameters and up to 4.6 times the time per epoch, so it is worth choosing when the task has long-range dependencies and optional when the signal is recent.

Future work. The experiments suggest five next steps: (1) pretrained word vectors and a bidirectional LSTM on EC, to test whether the gap to TF-IDF closes; (2) several seeds per configuration, to put confidence intervals on the RNN–LSTM differences, especially on ST; (3) a GRU as a third architecture, with two gates instead of three; (4) longer sequences, where the vanishing gradient should hurt the RNN much more, and a longer volatility horizon for ST, where memory should matter more; (5) market-wide volatility as an extra input for ST and a loss that fits the heavy tails better than the squared error.

13 Source Code Repository

All code of this assignment, the three notebooks with their saved outputs, the per-run result files and the sources of this report, is in a public GitHub repository:

`https://github.com/minorin2847/INTEL_SYS_DEV_2026`

The repository keeps each part of the work in its own place (Table 19); the large raw data files from Kaggle are not stored in it, only the notebooks that describe where they come from.

Table 19. Contents of the repository.

Path	Content
`00_rnn_lstm_concepts.ipynb`	elementary functions, 5-layer RNN and LSTM written by hand, checks against PyTorch and Keras, parameter counts, gradient maps (§2–§6)
`customer_review/`	EC: `notebook/ec_reviews.ipynb` (cleaning, tokenisation, baselines, four runs, evaluation), `results/` (one JSON per run and a comparison CSV), `data/` (raw file)
`stock_forecast/`	ST: `notebook/st_stocks.ipynb` (features, windows, baselines, four runs, evaluation), `results/`, `data/`
`report/`	the LaTeX source of this report, its figures (`images/`) and the script that turns the result files into the tables
`requirements.txt`, `README.md`	the Python packages and a guide to the repository

References

[1] S. Hochreiter and J. Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.

[2] Y. Bengio, P. Simard and P. Frasconi. Learning long-term dependencies with gradient descent is difficult. *IEEE Transactions on Neural Networks*, 5(2):157–166, 1994.

[3] R. Pascanu, T. Mikolov and Y. Bengio. On the difficulty of training recurrent neural networks. In *ICML*, 2013.

[4] F. A. Gers, J. Schmidhuber and F. Cummins. Learning to forget: continual prediction with LSTM. *Neural Computation*, 12(10):2451–2471, 2000.

[5] R. Jozefowicz, W. Zaremba and I. Sutskever. An empirical exploration of recurrent network architectures. In *ICML*, 2015.

[6] D. P. Kingma and J. Ba. Adam: a method for stochastic optimization. In *ICLR*, 2015.

[7] X. Glorot and Y. Bengio. Understanding the difficulty of training deep feedforward neural networks. In *AISTATS*, 2010.

[8] A. M. Saxe, J. L. McClelland and S. Ganguli. Exact solutions to the nonlinear dynamics of learning in deep linear neural networks. In *ICLR*, 2014.

[9] R. Cont. Empirical properties of asset returns: stylized facts and statistical issues. *Quantitative Finance*, 1(2):223–236, 2001.

[10] A. Paszke et al. PyTorch: an imperative style, high-performance deep learning library. In *NeurIPS*, 2019.

[11] F. Chollet et al. Keras, 2015. https://keras.io.

[12] PromptCloud. Amazon reviews: unlocked mobile phones. Kaggle dataset. https://www.kaggle.com/datasets/PromptCloudHQ/amazon-reviews-unlocked-mobile-phones.

[13] andrewmvd (Kaggle). S&P 500 stocks (daily updated). Kaggle dataset. https://www.kaggle.com/datasets/andrewmvd/sp-500-stocks.